Dissertation on

"Potential Fishing Zone Forecasting using Satellite Imagery"

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology
in
Computer Science & Engineering

UE20CS461A – Capstone Project Phase - 2

Submitted by:

Sai Janani C	**PES2UG21CS455**
Samantha Susan John	**PES2UG21CS463**
Saranga A Kulkarni	**PES2UG21CS479**
Ankit Hiremath	**PES2UG21CS075**

Under the guidance of

Dr. M. Farida Begam
Professor
PES University

June - Nov 2024

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled

'Potential Fishing Zone Forecasting using Satellite Imagery'

is a bonafide work carried out by

Sai Janani C	**PES2UG21CS455**
Samantha Susan John	**PES2UG21CS463**
Saranga A Kulkarni	**PES2UG21CS479**
Ankit Hiremath	**PES2UG21CS075**

In partial fulfilment for the completion of seventh semester Capstone Project Phase - 2 (UE20CS461A) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period June 2024 – Nov. 2024. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 7th semester academic requirements in respect of project work.

Signature Dr. M. Farida Begam Professor	Signature Dr. Sandesh B J Chairperson	Signature Dr. B K Keshavan Dean of Faculty

External Viva

Name of the Examiners	**Signature with Date**
1. _____	_____

2. _____ _____

DECLARATION

We hereby declare that the Capstone Project Phase - 2 entitled **"Potential Fishing Zone Forecasting using Satellite Imagery"** has been carried out by us under the guidance of Dr. M. Farida Begam, Professor and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester June – Nov. 2024. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES2UG21CS455 **Sai Janani C**
PES2UG21CS463 **Samantha Susan John**
PES2UG21CS479 **Saranga A Kulkarni**
PES2UG21CS075 **Ankit Hiremath**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. M. Farida Begam, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE20CS461A - Capstone Project Phase – 2.

I am grateful to the Capstone Project Coordinators, Dr. Farida Begam, Professor and Prof. Vandana M. Ladwani, Associate Professor, Department of Computer Science and Engineering, PES University for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Sandesh B J, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. B.K. Keshavan, Dean of Faculty, PES University for his help.

I am deeply grateful to Dr. M. R. Doreswamy, Chancellor, PES University, Prof. Jawahar Doreswamy, Pro Chancellor, PES University, Dr. Suryaprasad J, Vice Chancellor, PES University and Prof. Nagarjuna Sadineni, Pro Vice Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

The vastness and evolving nature of the ocean ecosystems present a significant challenge to the effective management of marine resources, particularly in fisheries. The sustainable management of marine fisheries depends on accuracy in the prediction of potential fishing zones, where favourable environmental conditions coincide with a high abundance of fish. The main obstacle lies in the creation of a system that estimates the best fishing zones by effectively analyzing satellite data. A feasible solution to this problem involves combining satellite imagery and the capabilities of Image Processing and Machine Learning, which we aim to leverage in this project to provide sustainable solutions to the management of fisheries by predicting potential fishing zones.

The world has witnessed the incredible potential of Machine Learning technologies. Despite this, there seems to be a visible absence in the use of such techniques within the fishing community. Through this project, we intend to integrate ML into this domain, improving the quality of fishery administration, by enhancing the accuracy of forecasting. Traditionally, the prediction of fishing zones relied only on a few oceanographic parameters i.e., Sea Surface Temperature and Chlorophyll-A concentration. The inclusion of various other indicators, apart from the conventional ones, like Sea Surface Height, Sea Salinity, Dissolved Oxygen, and others, would help refine the predictions made.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	**INTRODUCTION**	**01**
2.	**PROBLEM STATEMENT**	**02**
3.	**LITERATURE REVIEW**	**03**
	3.1 Provably Correct Peephole Optimizations with Alive	
	3.1.1 Introduction	
	3.1.2 Characteristics and Implementation	
	3.1.3 Features	
	3.1.4 Evaluation	
	3.2 Automatic Generation of Code Analysis Tools: The CastQL Approach	
	3.2.1 Introduction	
	3.2.2 Components	
4.	**PROJECT REQUIREMENTS SPECIFICATION**	
5.	**SYSTEM DESIGN**	
6.	**PROPOSED METHODOLOGY**	
7.	**IMPLEMENTATION AND PSEUDOCODE**	
8.	**RESULTS AND DISCUSSION**	
9.	**CONCLUSION AND FUTURE WORK**	

REFERENCES/BIBLIOGRAPHY

APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS

APPENDIX B USER MANUAL (OPTIONAL)

LIST OF FIGURES

Figure No.	Title	Page No.

LIST OF TABLES

Table No.	Title	Page No.

CHAPTER 1

INTRODUCTION

1.1 Fishing Industry

The Fishing industry and aquaculture are vital sources of food, employment, and income for a multitude of people. This sector primarily provides a livelihood to fishermen. Fish is an affordable and rich source of animal protein. It has great potential to increase the income and economic well-being of participants and prioritise its development sustainably and responsibly.

In India, fishing is one of the major economic sectors which accounts for 2.03% of the GDP. More than 30 million people in India depend on this industry. It is the world's 3rd-largest producer in terms of aquaculture output. As per the National Fisheries Development Board, exports from the fisheries industry have earned approximately $4.5 billion in a year alone. This basically shows that the Indian fisheries have a major role not only in terms of providing economic contribution, but also paving the way towards employment for millions.

The coastline of India has maritime states, two union territories, Andaman and Nicobar, and Lakshadweep islands. About 200 plus species are contributing to the marine fisheries. The most abundant fish that are easily available to catch are Indian mackerel, oil sardine, ribbon fishes, and lesser sardines dominating the catch. Indian oil sardines are the most fish landing.

Total marine fish landings in the country is about more than 35 lakh tonnes and in the coming years, it will increase drastically. The landing by various fishing catches occurs in 1200 plus designated locations along with fifty fishing harbours across the coastal regions of the country. Region-wise the South-West regions comprising states like Kerala, Karnataka, and Goa had the most fishing landing in recent years from (2020-2022), Tamil Naidu was in the top position in fish landing.

In the current fast-paced world, it is essential to manage marine resources for future generations. In particular, this means addressing problems such as overfishing. It has become an increasingly critical

global issue along with climate change which is posing a threat to the long-term sustainability of marine ecosystems worldwide. One way of doing this is by finding new techniques and approaches that will help fishermen while preserving ecological balance at the same time. For example, identifying Potential Fishing Zones (PFZ) identifies areas with high fish concentration and hence, fishermen benefit by increasing their catch which gives them help and tips on how they can maximise catching effort without causing too much harm to the environment.

This project will use advanced technologies including satellite imagery, image processing algorithms, and machine learning to come up with an accurate prediction about Potential Fishing Zones (PFZ) within a shorter timeframe than what is currently possible under other systems like ESSO- Indian National Centre for Ocean Information (INCOIS) issued Possible Fishing zone advisory system and to improve upon the existing models and current methodologies and overcome some limitations thereby providing a better solution.

CHAPTER 2

PROBLEM DEFINITION

This project will identify Potential Fishing Zones of seas and oceans using satellite images. The PFZ's will be predicted using image processing techniques and machine learning algorithms.

A Potential Fishing Zone is a region in the ocean with a high probability of finding commercially important fish or marine life. These zones are indicators for fishermen about the availability of fish needed to support their trade.

2.1 Problems Faced

- Traditionally, fish are found by the method of trial-and-error which is not viable as the ocean is constantly in a state of flux.
- Time taken is high and fuel efficiency is low when employing these traditional fishing methods.
- The uncertainty in catch is a problem and can lead to economic losses.
- Compliance with imposed regulations by governing bodies must be enforced to prevent overfishing and the destruction of the marine ecosystem.
- The existing solution in India is implemented by INCOIS (Indian National Centre for Ocean Information Services). They provide a daily PFZ advisory. However, they do not use machine learning algorithms for their predictions but use data analytics techniques instead.
- The existing solution by INCOIS considers only two parameters – SST and Chlorophyll-a concentration in their PFZ analysis, ignoring a variety of other parameters.

2.2 Impact

The project will impact the following groups:

- *Fishermen*: The PFZ advisory will benefit the fishermen directly by reducing fuel costs, and time spent searching, increasing their catch, and helping them operate sustainably.
- *Fisheries Management Authorities*: Their PFZ advisories could potentially be improved by the outcome of this project.
- *Environmental Conservation Organizations*: This project will promote only sustainable fishing practices.
- *Government Agencies* (such as INCOIS).
- *Research Institutions* (INCOIS, CMFRI)
- *Local Communities*: With economic prosperity comes an improvement in the quality of life in these fishing and coastal communities.

2.3 Background

The presence of marine life in a particular region is due to a plethora of reasons; it is not mere happenstance. Some of the main factors leading to a habitable environment for the various species of fish and other marine life are as follows -

- Sea Surface Temperature
- Sea Surface Height
- Chlorophyll-a Concentration
- Salinity
- Dissolved Oxygen Levels
- Upward Sea Velocity
- Precipitation

All of these parameters play a role in the habitability of a particular region of a water body and will be used to predict our PFZs.

Elaborating further on how these parameters affect marine life, we would like to highlight one of the most important parameters – *Sea Surface Temperature* (SST) – The optimal temperature for small pelagic fish ranges between 26 - 30 °C, and 30 - 32 °C for large pelagic fish.

Another large influence on marine habitats is the presence of *phytoplankton*. Phytoplankton occupy the lowest rung of the marine food chain, and small fish and crustaceans rely on it for sustenance. These primary consumers are subsequently eaten by larger fish, corals, and other predatory species. Thus, a food chain is formed with phytoplankton as the source. This illustrates how essential phytoplankton is to predicting a PFZ accurately.

Sea Surface Height within the range of 0.61 to 0.78 m and *Sea Salinity* within a 27% to 34% concentration gave a higher yield of pelagic fish.

Dissolved Oxygen Levels variably affect different species; however, the optimal range is between 5 and 20 ppm. Once a region's dissolved oxygen level falls below 3.7 mg/L fish tend to leave.

Upwelling is an important factor in marine environments as well. The nutrient-rich water near the ocean floor is cycled to the top, providing nourishment to the upper, nutrient-depleted levels where the fish are present. *Precipitation* has shown a correlation to a habitable marine ecosystem. However, extreme rainfall can negatively affect coastal marine life.

The parameters given above will be used to predict the presence of fish in our PFZ advisory. The optimal range for all these parameters will be recorded and extracted from the satellite images using image processing. The satellite images have been acquired from the Copernicus website, which is a free resource and will be used to implement this project.

Overall, this project is intended to better the lives of fishermen and their communities by using the latest technology and resources while also furthering advancements in existing research.

CHAPTER 3

LITERATURE SURVEY

3.1 Seasonal potential fishing zone model in the regional fisheries management of Indonesia (WPP-RI) 716 based on remote sensing satellite data

3.1.1 Objective

This paper estimates a fishing ground model employing satellite imagery in the fishery management area, WPP-RI 716, in Indonesia. The data for the years 2018–2019 have been collected. They have used edge detection techniques with SST being the key parameter.

3.1.2 Advantages

- The paper accounts for ocean currents and seasons.
- There is no usage of pre-existing satellite image processing tools.

3.1.3 Limitations

- There is no usage of machine learning techniques, which deteriorates performance.
- Obtaining precise and detailed information about fish populations from generalised fishing records is a difficult task.
- They have made multiple assumptions in selecting the environmental variables.

3.2 Fishing Area Prediction Using Scene-Based Ensemble Models

3.2.1 Objective

This paper attempts to identify Skipjack Tuna catch hotspots across the world. It uses statistical methods such as ensemble models consisting of two individual models - decision trees and generalised linear models with sea surface temperature, chlorophyll-a, and sea surface height as the environmental variables. It uses a density map to plot out the remote sensing data, which is calculated based on statistical techniques.

3.2.2 Advantages

- This paper has made successful predictions of fishing zones with an accuracy of 80%
- The scene-based model used with the individual models is highly effective.

3.2.3 Limitations

- The final prediction of the ensemble model depends on the prediction of the individual models, which in this case offers only two options. So the impact of the individual models' shortcomings and biases on the model is huge.
- The data used is from 2016-2017. This limited data used to train the model results in the model finding it hard to predict fishing zones over a wider time range. This lack of generalisation proves to be a major limitation of this paper.

3.3 Forecasting Albacore (*Thunnus alalunga*) Fishing Grounds in the South Pacific Based on Machine Learning Algorithms and Ensemble Learning Model

3.3.1 Objective

The paper forecasts different grades of Albacore fishing grounds in the South Pacific Ocean. They have collected data from the years 2009 to 2019 for doing so. The paper leverages the power of ensemble learning models and has used five individual machine learning models, namely Random Forest, SVM, KNN, XGboost, and Gaussian Processes.

3.3.2 Advantages

- There is an effective prediction of high-yielding fishing grounds.
- The high accuracy of the forecasting models, results in high performance metrics.
- The paper has identified key features in the usage of every model and its correlation with the data.

3.3.3 Limitations

- There is inaccurate forecasting in the boundaries of high-yield and low-yield areas, which can lead to the detrimental pattern of overfishing, leading to a sustainability concern.
- One of the metrics used is catch per unit effort. This does not fully capture the essence of fishing grounds.

3.4 HE-DFNETS: A Novel Hybrid Deep Learning Architecture for the Prediction of Potential Fishing Zone Areas in Indian Ocean Using Remote Sensing Images

3.4.1 Objective

The paper uses a hybrid ensemble model to predict PFZs in the Indian Ocean. The remote sensing images consisting of SST and chlorophyll-a parameters are used. They have used a novel ensembled double-tier convolutional network that handles the processing of images. The predictions from the extracted data are made using an LSTM along with a novel flitter-bat optimization algorithm.

3.4.2 Advantages

- Compared to other proposed frameworks, the high-performance metric values give this paper an edge over multiple others.
- All the models provided are originally curated by the authors. This makes them novel solutions, with no prior implementation elsewhere.

3.4.3 Limitations

- The ensemble convolutional layers lead to an extremely unfeasible computational complexity.
- LSTM performance degrades with larger datasets.

3.5 Mapping potential fishing zones based on sea surface temperature and Chlorophyll-A in the Waters of Aceh Besar, Indonesia

3.5.1 Objective

This paper has utilised geographic information systems (GIS) to map out the potential fishing zones based on SST and chlorophyll-a concentrations. The analysis starts with extracting data from MODIS satellite imagery. Processing of the information is done using already existing software tools like SeaDAS and ArcGIS, and by overlaying SST and chlorophyll-a distribution images to identify the potential fishing zones.

3.5.2 Advantages

- There is an integration of primary data collected by interviewing fishermen and secondary data from satellite imagery.
- The paper validates the estimated potential fishing zones with data from actual fishing operations conducted by local fishermen.

3.5.3 Limitations

- There is no usage of machine learning techniques which reduces the accuracy of the prediction.
- The usage of pre-existing tools to process satellite images makes it difficult to control finer aspects of the image processing.

CHAPTER 4

PRODUCT REQUIREMENTS SPECIFICATION

4.1 Introduction

4.1.1 Project Scope

Regional Focus - Excludes oceans beyond the specified geographical area and inland water bodies.

Environmental Specification - This project will focus on dynamic marine environments, and on the extensive ocean ecosystem rather than inland water bodies such as rivers or ponds, since oceans present unique challenges and opportunities when it comes to sustainable fishing practices.

4.1.2 Motivation

Despite the huge potential of Machine Learning (ML) technologies, there exists a noticeable lack of large-scale adoption within the fishing sector. By integrating ML into fisheries management, this project aims to revolutionise the fishing industry by leveraging advanced algorithms to enhance the precision of forecasting.

Furthermore, the traditional approach to fishing zone prediction often relies only on a couple of indicators, i.e., Sea Surface Temperature (SST) and Chlorophyll-a conc. This project will consider a variety of environmental parameters apart from the traditional ones, such as Sea Surface Temperature (SST), Chlorophyll-a Conc., Sea Surface Height (SSH), Salinity, Dissolved Oxygen Level, Phytoplankton Growth, and Upward Sea Velocity, providing a comprehensive and accurate prediction of potential fishing zones.

4.2 Product Perspective

4.2.1 Product Features

1. Optimal Fishing Zone Prediction

   Prediction of high abundance areas by training an ML Model by feeding it data acquired by an image processing model that extracts data from satellite images.

2. Usage of Multiple Indicators

   Usage of parameters other than SST and Chlorophyll-a not only helps in improving the efficiency of the prediction model, but also adds to the novelty of the project.

3. Consideration of Prohibited Fishing Zones

   The project also has to take into account local laws and regulations, and mustn't flout them by predicting zones in prohibited areas.

4.2.2 User Classes and Characteristics

1. **Primary Stakeholders**

   a. Fishermen

      Rely on accurate information about PFZs for their livelihood.

      May vary in terms of technical capability. Some may need user-friendliness while some are okay with tabular views.

   b. Fisheries Management Authorities

      Responsible for regulating fishing activity to ensure compliance with regulations. May use PFZ data to develop efficient management strategies. Can also use fishing data for policy-making purposes.

c. Environmental Conservation Organizations

May utilise the PFZ data to identify areas of ecological significance. They can also use the visualisations to support their conservation efforts.

d. Developers

Responsible for designing, developing, and maintaining the software or platform used to access and analyse fishing zone data.

2. Secondary Stakeholders

a. Government Agencies

Regulatory bodies responsible for enforcing fishing regulations and maritime laws. May require access to PFZ data to support law enforcement and compliance activities.

b. Research Institutions

Conduct scientific research on marine ecosystems, fisheries management, and environmental conservation. Utilise fishing zone data for academic research, modelling, and analysis to further scientific understanding.

c. Technology Providers

Organisations that supply the technology infrastructure, such as satellite imagery, data processing tools, etc. can collaborate with developers and stakeholders to optimise existing technology for fishing zone prediction.

3. Tertiary Stakeholders

a. Local Communities

Residents living in the coastal regions may benefit from improved fisheries management practices.

b. Tourism Industry

   Businesses involved in diving, snorkelling, etc. rely on healthy marine ecosystems and sustainable fishing practices.

4.2.3 Operating Environment

The system will be designed with ease of use in mind. Hence, it can operate on standard hardware platforms that have access to the internet, and a modern web browser to be able to interact with the user.

4.2.4 General Constraints, Assumptions and Dependencies

1. Data Limitations

   Factors such as cloudiness, unavailability or inconsistency in the remote sensing data can affect the reliability of the model.

2. Regulatory Policies

   Regulatory policies for the target area could hinder the availability of datasets for the model. It is also something the model will have to keep up with.

3. Satellite Data Resolution

   The remote sensing data may be limited in terms of spatial and temporal resolutions, affecting the quality of predictions.

4.2.5 Risks

1. Model Assumptions

   Assumptions made in the modelling process, such as the relationship between sea surface parameters and fish distribution, may not always hold true in real life scenarios, leading to potential inaccuracies in the results.

2. Validation Issues

   Difficulty in acquiring high quality ground-truth data or catch information for validating the model.

3. Seasonal Variability and Environmental Impact

   Variability in seasonal patterns and environmental conditions may impact the consistency and stability of potential fishing zones over time, requiring continuous monitoring and updates.

4.3 Functional Requirements

4.3.1 Image Preprocessing

Includes conversion of .nc files into .png images and pre-processing of images to enhance image quality, correct for atmospheric effects and remove noise and artefacts. Prepares the images for data extraction.

4.3.2 Data Extraction

Image processing techniques alongside Machine Learning to greatly enhance the accuracy and efficiency of extraction of data from the satellite images.

4.3.3 ML Model

Preparation of training, testing and validation datasets by cleaning the numerical data acquired. Training of models based on the dataset to learn patterns and extract features. Model selection to pick the best ML models for regression and classification. Evaluation of models is done based on metrics such as accuracy, precision, recall, F1 score, etc.

4.3.4 Fishing Zone Prediction

Usage of the ML model and acquired dataset to forecast future fishing zones.

4.3.5 Data Visualisation

Visualisation of the final PFZs on a map for easy readability. Interactive interface for users to explore and analyse the predictions. Presence of a tabular view of the data as well.

4.4 External Interface Requirements

4.4.1 User Interfaces

The system will feature a web-based user interface focusing on ease of use and intuition. The UI will adopt a map-centric design, presenting users with a comprehensive view of potential fishing areas by overlaying predicted fishing zones on relevant oceanographic data. User-friendly controls will be incorporated to enable filtering options, enhancing the customization and precision of fishing zone predictions. Additionally, the interface will provide clear visualisation of prediction confidence levels, allowing users to assess the reliability of the generated fishing zone predictions with ease.

4.4.2 Hardware Requirements

1. Computing Devices

   For development, desktop computers, laptops, or servers capable of handling image processing tasks. Should have sufficient processing power, memory, and storage capacity to process and analyse satellite images effectively. The product will be compatible with popular operating systems such as Windows, MacOS, or Linux.

2. Graphics Processing Units (GPUs)

   Required to accelerate image processing and ML model training tasks, especially for complex algorithms. Compatible GPUs with CUDA or OpenCL support for parallel processing and faster computation.

3. Data Storage

   High-capacity storage devices or servers are needed to store satellite images and processed data. Cloud storage solutions for data redundancy and scalability.

4. Network Infrastructure

   High-speed internet connection for downloading satellite images and accessing remote data repositories.

5. Interface Devices

   Monitors, keyboards, and mice for interacting with the image processing software and the web UI.

6. Communication Protocols

   TCP/IP for communication between devices and data transmission over the internet. File transfer protocols such as FTP or SFTP for exchanging large satellite image files securely. RDP, VNC for accessing and controlling image processing systems remotely.

4.4.3 Software Requirements

1. Operating Systems - Compatible with Windows, macOS, and Linux distributions.
2. Tools - Python programming language for machine learning algorithms.
3. ML Libraries - Python libraries like TensorFlow, PyTorch, and scikit-learn.
4. Source Control - Version Control Systems (VCS) such as Git.

4.5 Non-Functional Requirements

4.5.1 Performance Requirements

1. Processing Speed

   The system should be able to process and analyse large volumes of satellite imagery data within a reasonable timeframe to provide timely fishing zone predictions.

2. Scalability

   The system should be capable of scaling up to handle increased data volume and user demand without significant degradation in performance.

3. Response Time

   The system should respond to user queries or requests for fishing zone predictions within a predefined time frame to ensure a smooth user experience.

4.5.2 Safety Requirements

1. Data Integrity

   The system should ensure the integrity and reliability of data used for fishing zone prediction to prevent inaccurate or misleading results.

2. Regulatory Compliance

   The system should comply with relevant safety regulations and standards governing the use of satellite imagery and fisheries management practices.

CHAPTER 5

SYSTEM DESIGN

5.1 Current System

A system by the ESSO - Indian National Centre for Ocean Information Services (INCOIS) called the Potential Fishing Zone (PFZ) Advisory exists, which has the following primary components -

- Utilisation of Sea Surface Temperature (SST) and Chlorophyll
- Remote Sensing data from NOAA-AVHRR, Eumetsat's Met-Op series satellites, Oceansat-II and MODIS Aqua satellites is used.
- Advisories are generated for 14 regions of the Indian coastline.

Some of the advantages of the INCOIS PFZ advisory are as follows:

- The zones predicted due to the PFZ Advisory have a good accuracy. A lot of pelagic fish is found within the zones, which proves important commercially.
- There is a reduction in searching time, resulting in savings on fuel and human effort.
- The PFZ Advisory, due to push from the government, has high reachability among the fishing community and has been used for a few years now.

Some of the disadvantages of the INCOIS PFZ advisory are as follows:

- No usage of machine learning or modern algorithms.
- Prediction data for a particular day is only available that day, and not beforehand, making it inconvenient for fishermen to plan routines beforehand.
- Since the prediction depends on satellite data of that day only, presence of cloud cover and other hindrances leads to inaccurate predictions or no predictions at all.
- The PFZ maps are hard to access and very difficult to comprehend.

5.2 Design Considerations

5.2.1 Design Principles

1. Incorporation of Modern Technology

   Usage of modern ML algorithms in the system to improve PFZ prediction accuracy and overcome the limitations of statistical or manual interpretation.

2. User-Centric Design

   The system's primary focus will be on user-friendliness and ease of use, with intuitive navigation and clear visualisations.

3. Reliability and Trustworthiness

   The system will prioritise reliability and trustworthiness by ensuring accuracy and consistency of the PFZ predictions.

5.2.2 Design Guidelines

1. Usability and Accessibility

   The system will focus on ensuring the usability and accessibility of the system by making it easy for fishermen to access and interpret PFZ predictions.

2. Performance Optimization

   The system aims at optimising performance to ensure quick response times to ensure better access to PFZ predictions.

3. Scalability and Flexibility

   The system must be able to accommodate any future growth and changes in requirements.

5.2.3 Design Goals

- Development and usage of Machine Learning to predict Potential Fishing Zones.
- Development and usage of an efficient satellite image processing technique to streamline the prediction of Potential Fishing Zones.
- Development and usage of an easy-to-use user interface for the proposed model. The user interface will include an easy-to-understand graphical interface with tabular visualisations which improve upon the current system by a large amount.

5.2.4 Architecture Choices

Initially -

1. Acquire the image dataset
2. Pre-process the image dataset
3. Process the image dataset using a 3rd party GIS software such as ArcGIS API to acquire parameter values
4. Train, test and validate the ML model based on the acquired numerical data
5. Acquire the Potential Fishing Zone (PFZ) results using the developed ML model
6. Plot the result

Currently -

1. Acquire the image dataset
2. Pre-process the image dataset
3. Run the images through an ML model to separate out the concerned parameter values
4. Extract the values
5. Train, test and validate the prediction ML model based on the acquired numerical data
6. Acquire the Potential Fishing Zone (PFZ) results using the developed prediction ML model
7. Visualise the result

Pros:

- Comprehensive approach covering data preprocessing, image processing algorithm development, model training and visualisation.
- Utilises advanced techniques like machine learning for accurate PFZ prediction.
- Provides visualisation through an intuitive website interface.

Cons:

- Requires substantial computational resources for image processing and machine learning tasks.
- Complexity may lead to longer development and deployment times.
- Difficulty in acquiring a comprehensive validation dataset.

In conclusion, the initial plan was to use a 3rd party satellite image processing software to extract numerical data from image data, but the current plan has advanced such that the image processing is also part of the planned model.

5.2.5 Constraints, Assumptions and Dependencies

Some of the requirements are -

1. Interoperability Requirements

   Interoperability with various sources for satellite data must be ensured for smooth workflow.

2. Interface/Protocol Requirements

   Interfacing of the system with various data formats such as from .nc in the raw format to .png to process the images to getting numerical values in a .csv format.

3. Data Repository Requirements

   Usage of a suitable database such as a columnar database to increase column query efficiency, since each column is stored as an individual file.

Some constraints are -

1. Performance Issues

   Since the ML models are computationally expensive, improving the performance of the system is one of the top priorities. Query efficiency also comes into picture, since our dataset needs to be queried, pre-processed and trained, and the results must be stored back in the database.

2. Data Limitations

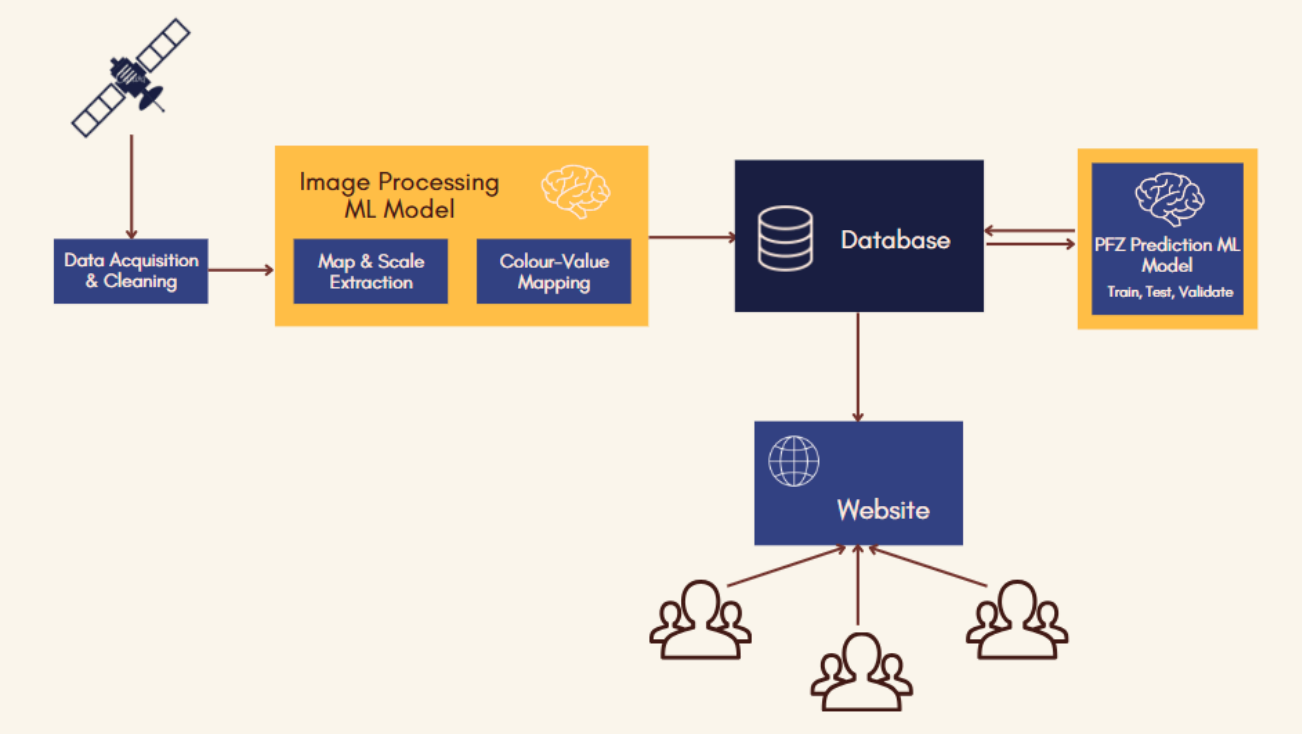
   Inaccuracies or limitations in the remote sensing data such as cloudiness, unavailability or inconsistency could affect the reliability of the identified potential fishing zones.

3. Validation Data Acquisition

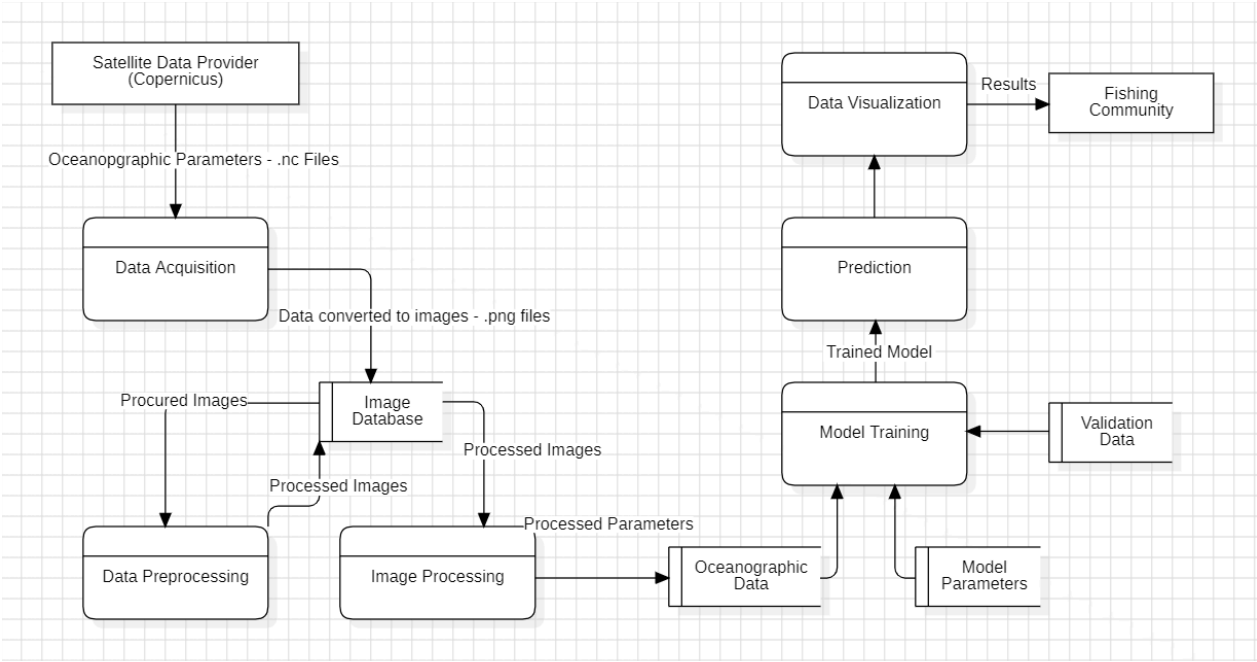
   Challenges in obtaining validation data from the primary data source, but alternative methods are being actively explored to address this issue.

5.3 High Level System Design

5.3.1 Architecture Diagram

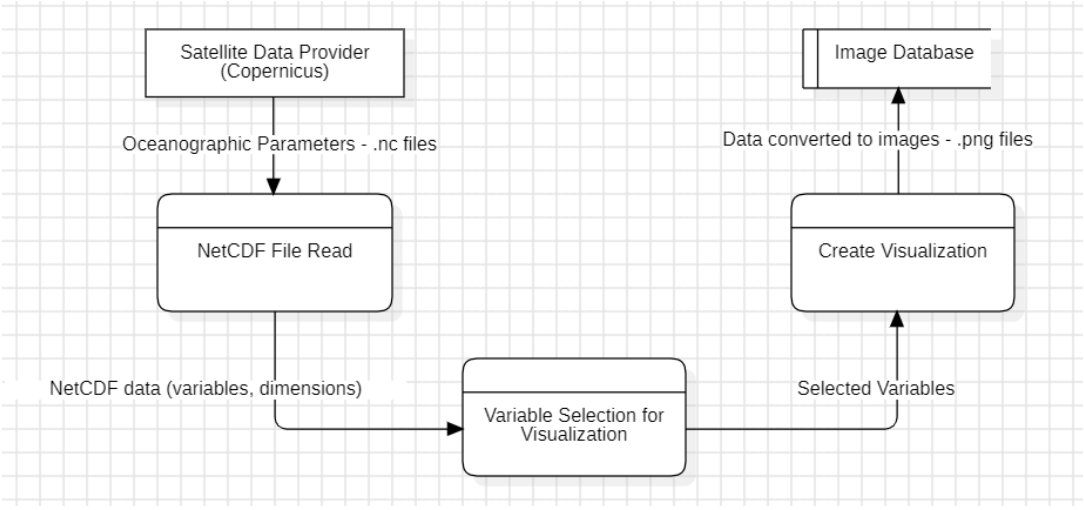


5.3.2 Level-0 Data Flow Diagram

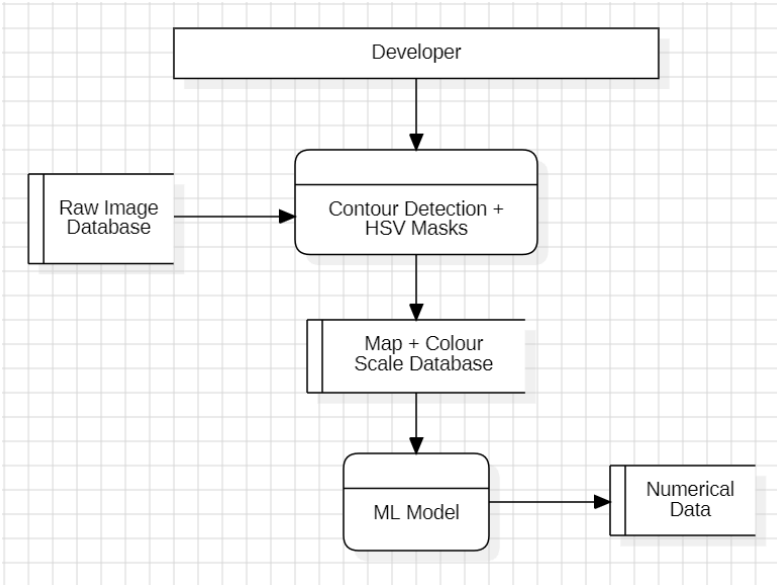


5.3.3 Level-1 Data Flow Diagrams

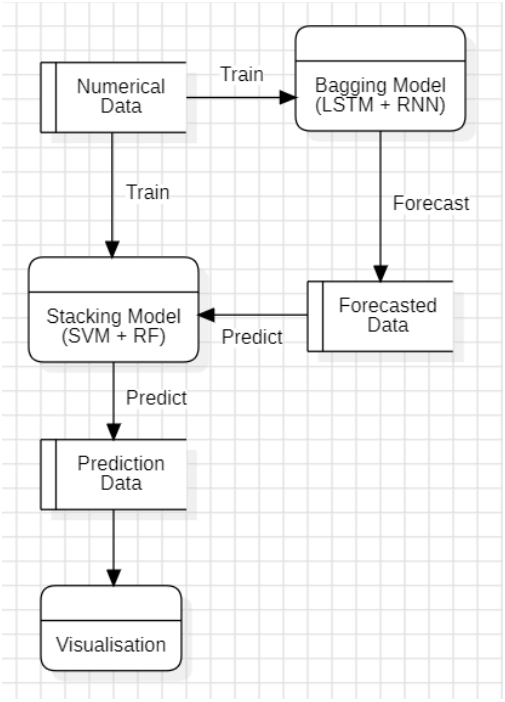
1. Image Preprocessing



2. Image Processing

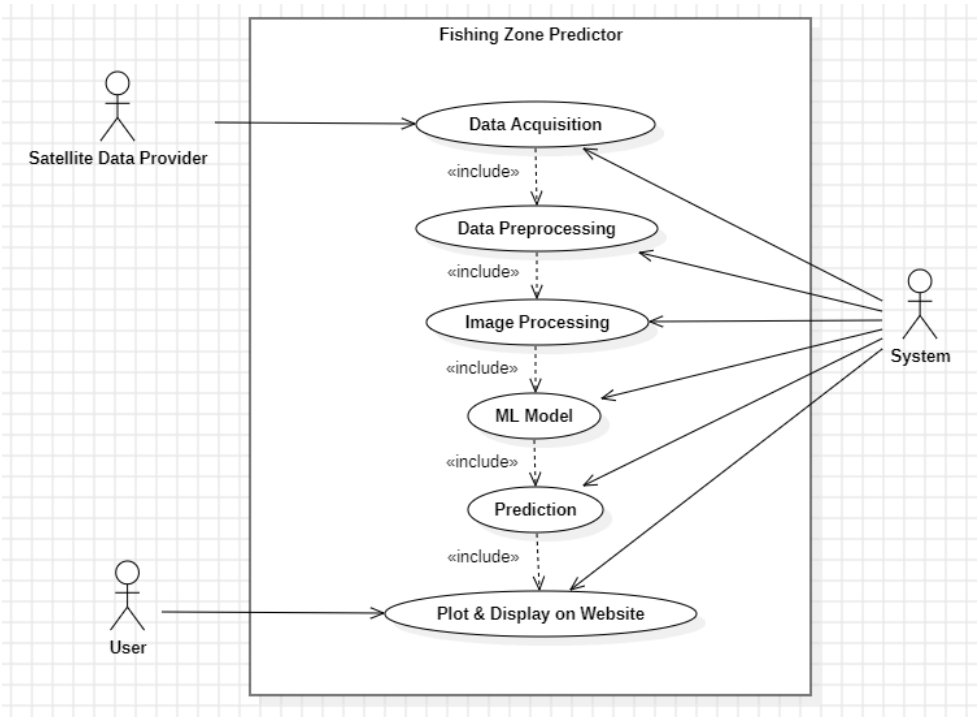


3. Machine Learning

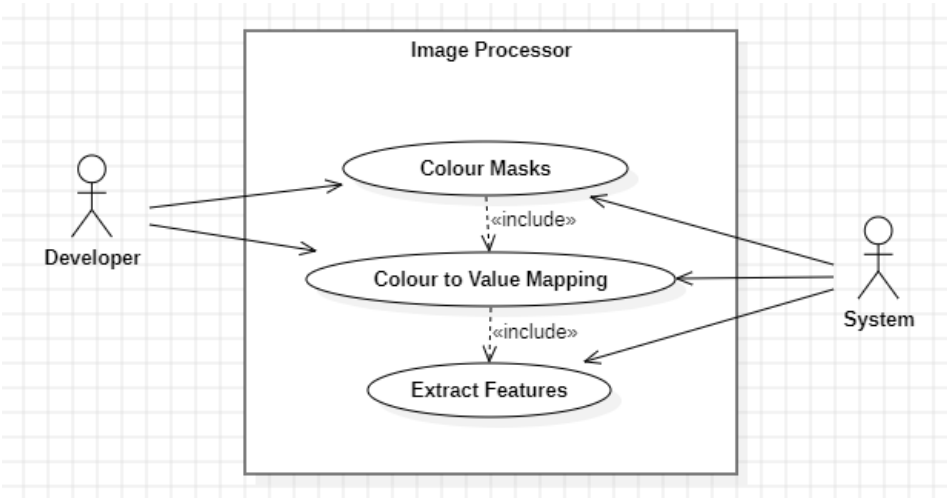


5.3.4 Use Case Diagrams

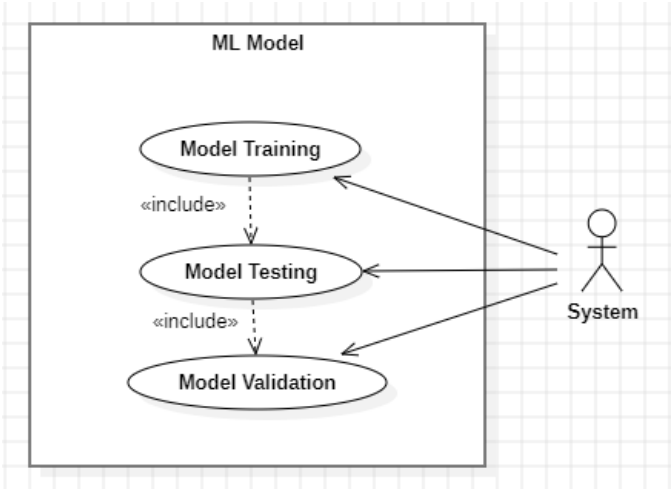
1. High Level View



2. Image Processor



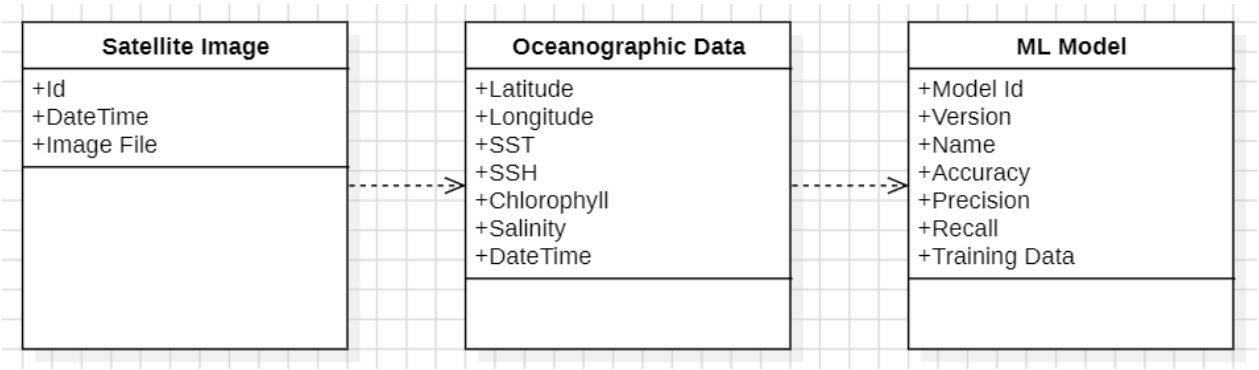
3. ML Model



5.3.5 Database

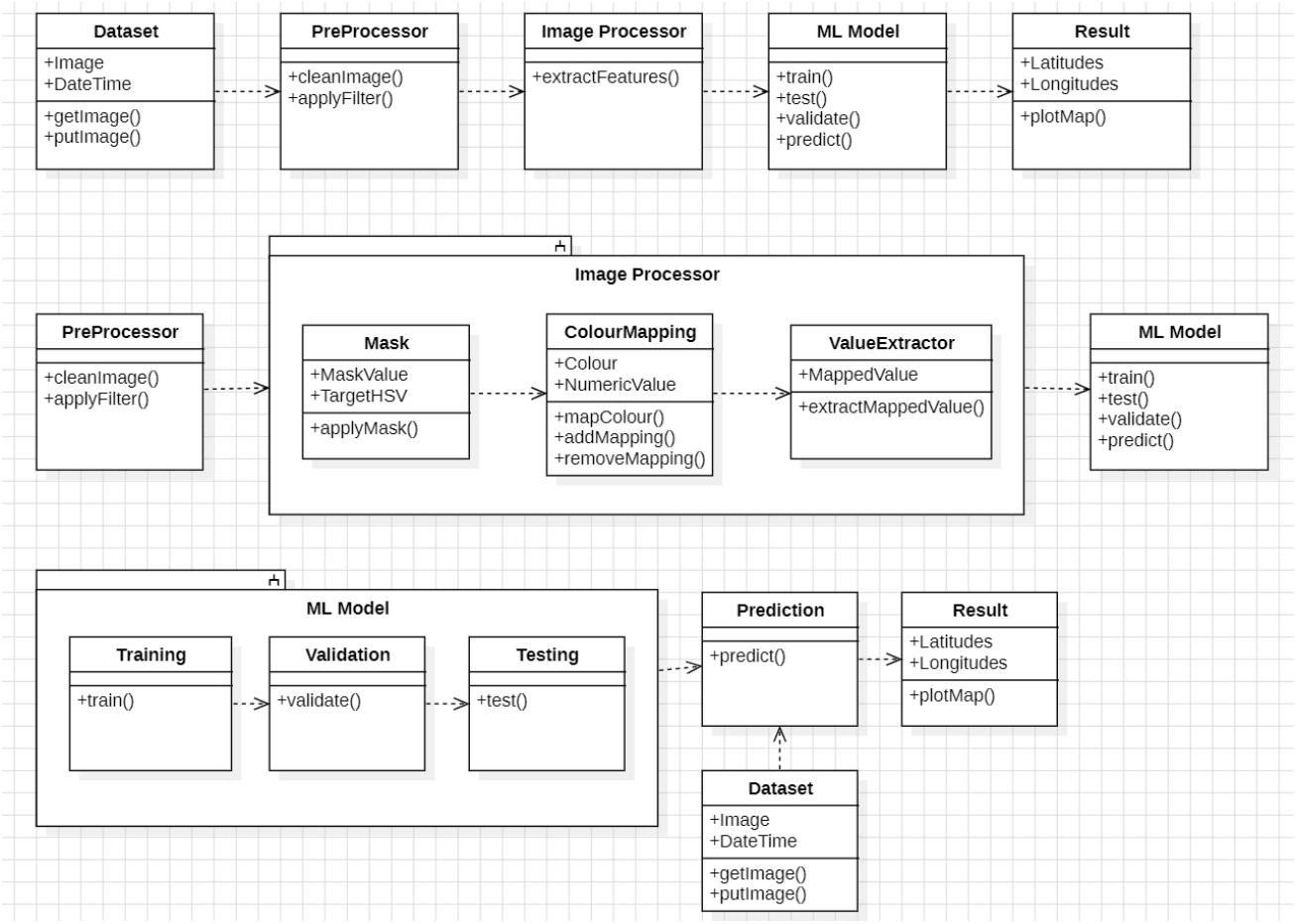
Columnar databases would suit this project the best, since columnar queries are more important than row-based queries since attributes will have to be queries all at once to train the model, and be updated by the image processor.

Entity Class Diagram -



5.4 Design Description

5.4.1 Master Class Diagram



The main class consists of 2 subsystems, namely -

1. Image Processor
2. Prediction ML Model

5.4.2 Reusability Considerations

Reusability in the project is introduced in the form of -

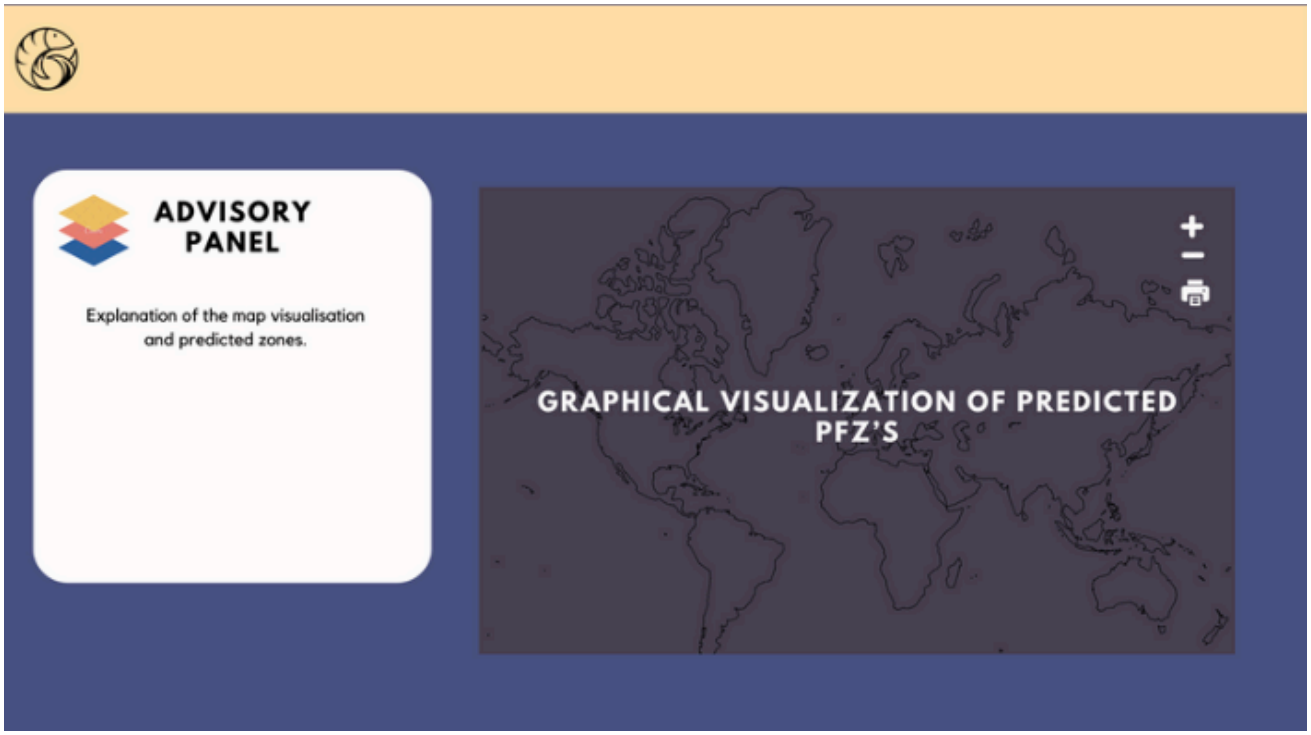
1. Image Processor – a common image processing model can be used to process the various images for each parameter.
2. Image Preprocessor – a common image preprocessing algorithm can be used to preprocess the various images for each parameter.
3. ML Model – the trained ML model can be used many times to predict PFZs before needing to be retrained on fresher data.

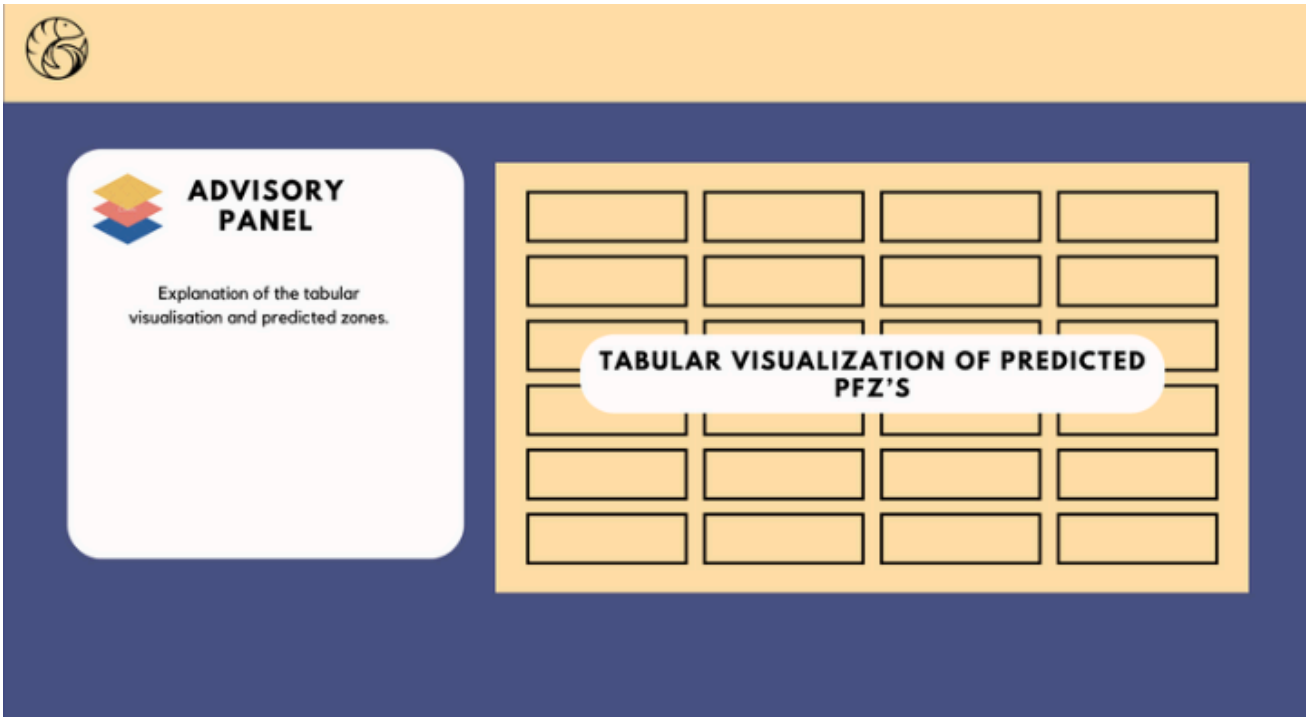
5.5 User Interface Diagrams

A simple website is planned. The website will not need any user authentication since it is for everyone's access. It is styled in a simple, advisory format.

The PFZ advisory will be presented in both map and tabular format.


POTENTIAL FISHING ZONES
PREDICT


ADVISORY PANEL
Explanation of the map visualisation and predicted zones.
GRAPHICAL VISUALIZATION OF PREDICTED PFZ'S



5.6 Help

A comprehensive documentation for the entire system is planned, along with steps on how to generate the required PFZ map and table.

5.7 Design Details

5.7.1 Novelty

The novel aspects of the project include -

- Usage of ML to process the satellite images
- Usage of ML to predict the PFZs
- Consideration of multiple key parameters to predict the PFZs

5.7.2 Innovativeness

The innovativeness of the project lies in –

- Usage of GIS-based Satellite Image data
- Usage of machine learning extensively to both process the satellite data and predict the PFZs
- Emphasis on sustainability

5.7.3 Interoperability

Interoperability is ensured through the usage of various APIs to fetch the satellite image data and the connection between the image preprocessor, image processor and the ML model.

5.7.4 Performance

Despite relying on large datasets to train and deploy the ML models, there is a great emphasis on performance for the project to ensure a seamless user experience. This is further supported by the usage of Python for model training, and the usage of a columnar database for querying.

5.7.5 Reliability

Reliability of the project is ensured by the following -

1. Reliable Data Source - The satellite data is being acquired from Copernicus which is an EU programme aimed at developing European information services based on satellite Earth Observation and in situ (non space) data.

2. Model Validation - The developed models will be validated against robust validation datasets to ensure high accuracy in the prediction of PFZs

5.7.6 Maintainability

Maintainability of the projects is ensured by -

- The usage of Python, as it is easy to use and maintain
- Periodic updates to the model to keep up with the latest ecological trends

5.7.7 Legacy to Modernization

The system aims to be a direct upgrade over the existing PFZ advisory system. The aim is to provide a more accurate, easy to use system for the benefit of the fishing community.

5.7.8 Resource Utilisation

The system uses satellite image data provided by Copernicus and INCOIS from various satellites such as Copernicus Sentinel-3A and -3B, MODIS, NOAA, etc.

5.8 Low Level System Design

5.8.1 Design Considerations

Some design considerations are -

1. Data Volume and Storage

   Handling large amounts of satellite data requires optimised storage solutions and efficient retrieval mechanisms.

2. Computational Power

   Satellite image processing and machine learning models will require significant computational resources, such as GPU or high-performance computing clusters.

3. Accuracy and Precision

The prediction models should be designed to prioritise accuracy in identifying potential fishing zones, balancing computational efficiency with the complexity of the algorithms.

4. Integration with External Systems

   The design should ensure smooth integration with external data sources, such as oceanographic data repositories, and with tools used by fisheries for decision-making.

5.8.2 Design Assumptions

The following assumptions have been made during the design process:

1. Data Availability

   It is assumed that satellite data (e.g., Sea Surface Temperature, chlorophyll-a concentration, Sea Surface Height) will be regularly updated and available in a consistent format.

2. Pre-Processed Data

   Raw data may be assumed to be pre-processed for noise reduction and calibration, allowing the system to focus on analysis rather than data cleaning.

3. User Proficiency

   It is assumed that the end-users, such as fishery managers, have sufficient technical expertise to interpret the results and make informed decisions.

5.8.3 Design Dependencies

The following external and internal dependencies are identified for the successful functioning of the system:

1. Satellite Data Providers

   The system is dependent on external satellite data providers to supply accurate, up-to-date oceanographic data.

2. Machine Learning Libraries

   Dependencies include machine learning frameworks (e.g., TensorFlow, PyTorch) for building and training prediction models.

3. Data Storage Systems

   The system relies on storage services or local servers to store and retrieve large datasets efficiently.

4. APIs for Oceanographic Data

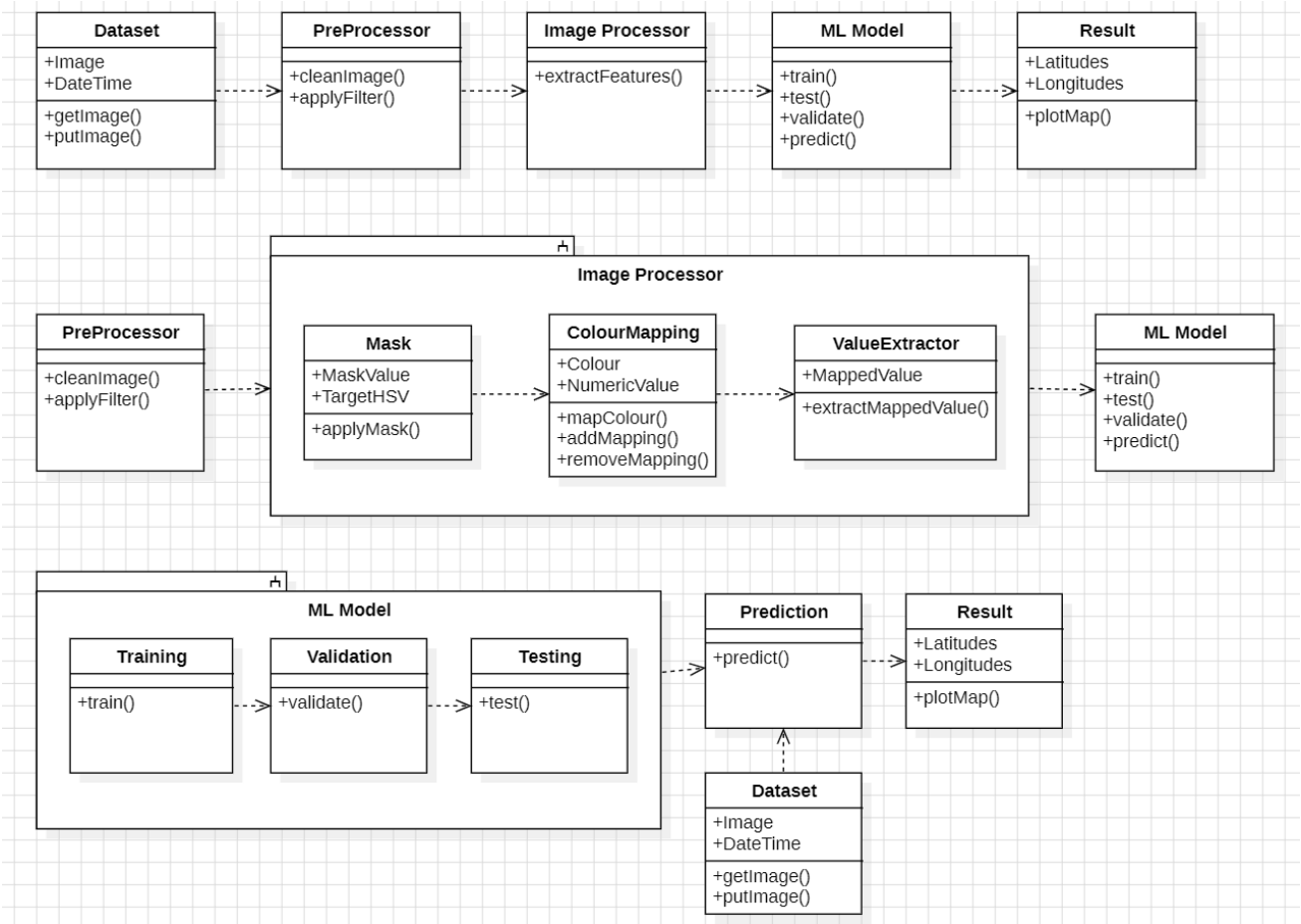
   The system depends on APIs to access real-time oceanographic parameters such as salinity and dissolved oxygen levels.

5. User Interface Platforms

   The performance of the system is dependent on the front-end platforms used to display the predictions and reports to users.

5.8.3 Design Description

5.8.3.1 Master Class Diagram



5.8.3.2 Image Processor

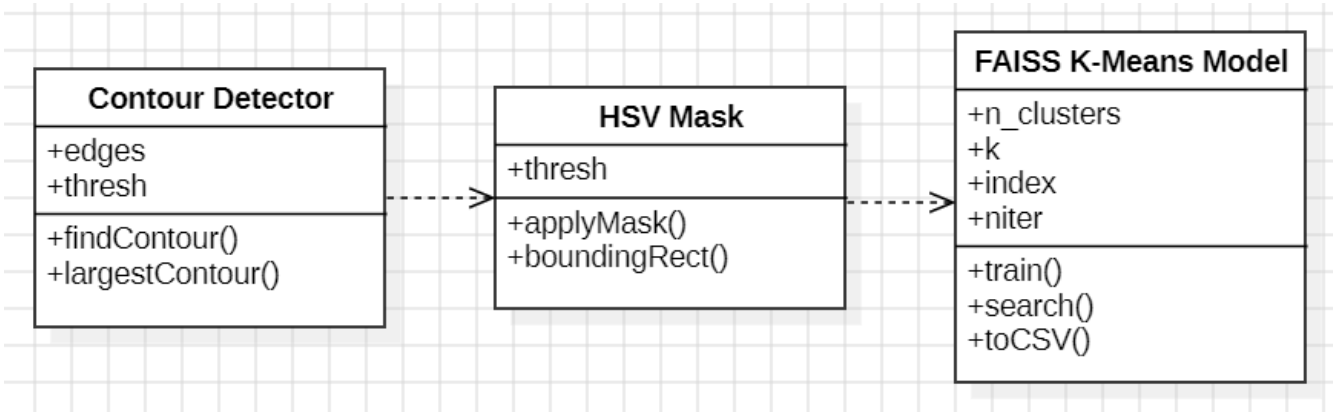
5.8.3.2.1 Description

The Image Processor class is responsible for -

1. Conversion of .nc files to .png files
2. Cleaning and preparation of raw images for data extraction
3. Extraction of numerical data from the satellite images

The output of this class is a dataset containing numerical values extracted from the satellite images, ready to be used in the training of the Prediction ML Model class.

5.8.3.2.2 Class Diagram



1. Contour Detector

   This class is responsible for contour detection in the map image, to help define the regions of the map and the colour scale in the satellite image. The attributes are *edges*, which is the result of a Canny filter, and *thresh*, which defines the threshold responsible for the result of the contour detection. The methods are *findContour()* which simply finds the contours in the image, and *largestContour()*, which finds the maximum contour.

2. HSV Mask

   This class is responsible for accurately determining the boundaries of both the map and the colour scale regions in the satellite images. The attribute is *thresh*, which defines the upper and lower HSV value bounds. The methods are *applyMask()*, which simply applies this HSV mask, and *boundingRect()* which draws a bounding rectangle around the target areas.

3. FAISS K-Means Model

   This class is responsible for extracting the numerical data from the satellite images by interpolating every map pixel's value, mapping it to its respective colour on the scale. The attributes are *n_clusters*, which is the number of clusters, *k*, which is the number of nearest neighbours, *index*, which is the FAISS index created and *niter*, which is the number of iterations for the model. The methods are *train()*, which trains the ML Model, *search()* which is responsible for mapping the nearest neighbours in the FAISS index and *toCSV()*, which converts the map data into a CSV format.

5.8.3.3 Prediction ML Model

5.8.3.3.1 Description

The Prediction ML Model class is responsible for finding the PFZ using the numerical data extracted from each satellite image. This is done using an ensemble model consisting of -

1. Bagging Ensemble Model

   The bagging model is responsible for forecasting of data that can then be used for PFZ classification. It consists of -

   a. LSTM

   b. RNN

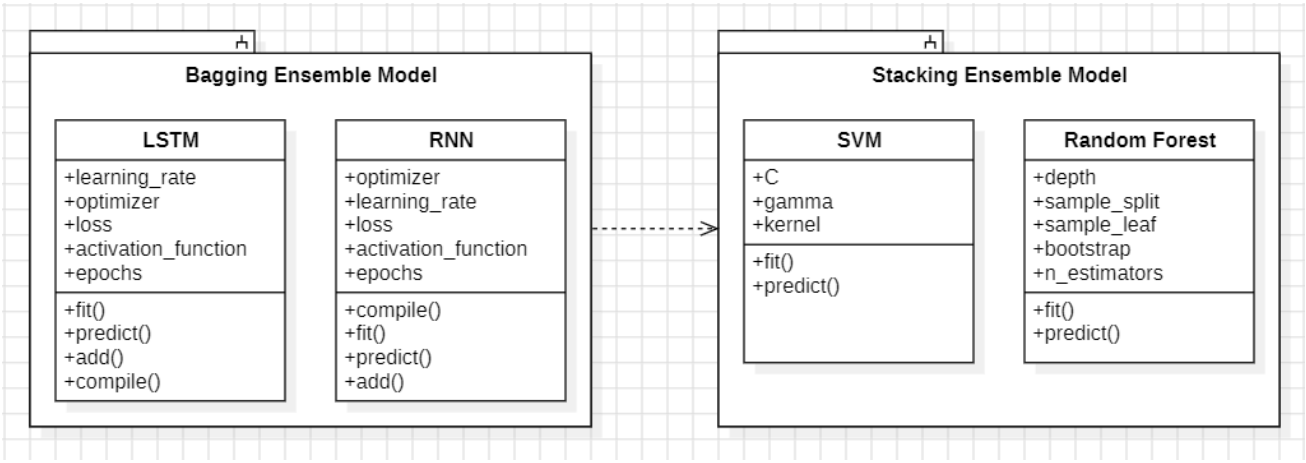
2. Stacking Ensemble Model

The stacking model is responsible for the classification of the numerical data into PFZs that can then be visualised. It consists of -

a. SVM

b. Random Forest

5.8.3.3.2 Class Diagram



1. LSTM

   This class is part of the Bagging Ensemble subsystem, and is responsible for forecasting the parameter values for the determined time period. The attributes are *learning_rate*, *optimizer*, *loss* function, *activation_function* and *epochs*. The methods are *fit()*, which trains the model, *predict()*, which gives the forecast output, *add()* and *compile()*, for the tuning of the model.

2. RNN

   This class is part of the Bagging Ensemble subsystem, and is responsible for forecasting the parameter values for the determined time period. The attributes are *learning_rate*, *optimizer*, *loss* function, *activation_function* and *epochs*. The methods are *fit()*, which trains the model, *predict()*, which gives the forecast output, *add()* and *compile()*, for the tuning of the model.

3. SVM

   This class is part of the Stacking Ensemble subsystem, and is responsible for classification of forecasted data into PFZs. The attributes are *C*, which is the regularisation parameter, *gamma*, which is the kernel coefficient and *kernel* type. The methods are *fit()*, which trains the SVM model and *predict()*, which gives the classification output.

4. Random Forest

   This class is part of the Stacking Ensemble subsystem, and is responsible for classification of forecasted data into PFZs. The attributes are *depth*, which determines the number of nodes, *sample_split* and *sample_leaf* which determine the requirement for a split and to be leaf nodes respectively, *n_estimators* which is the number of trees in the random forest and *bootstrap*, which is to bootstrap the dataset. The methods are *fit()*, which trains the random forest model and *predict()*, which gives the classification output.

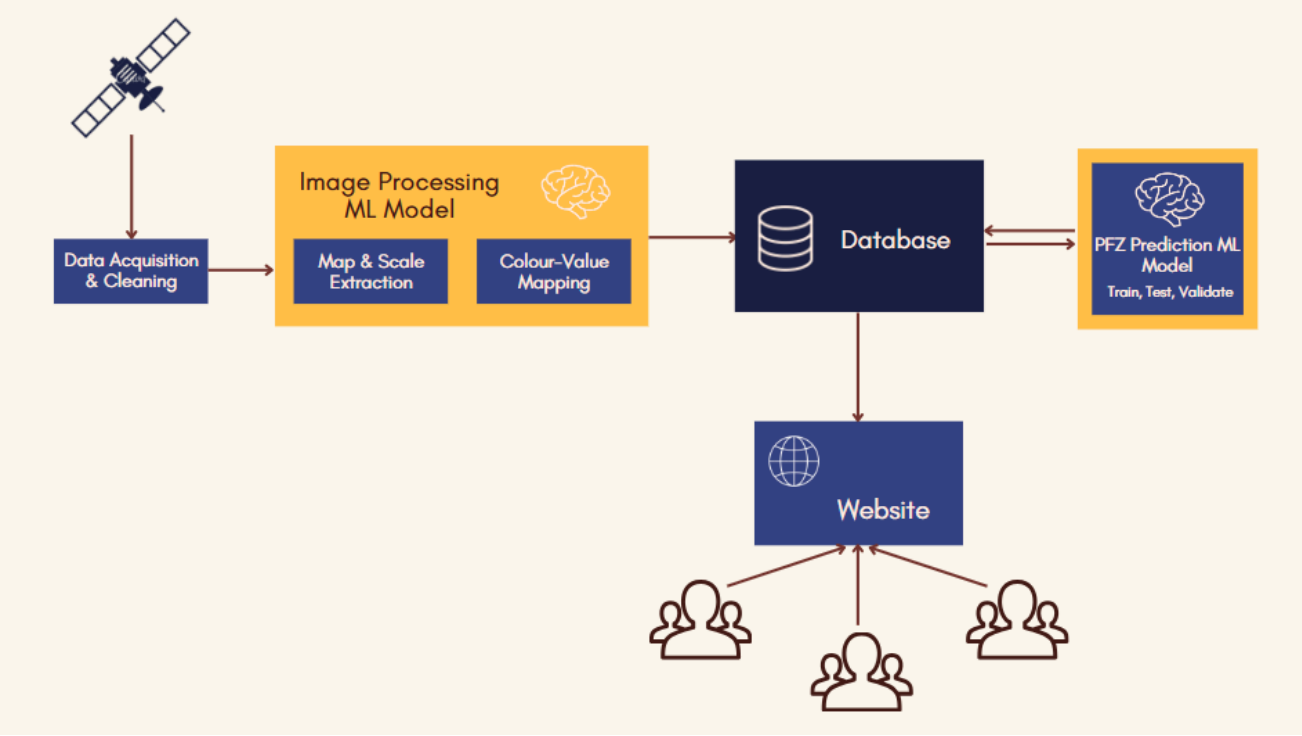
CHAPTER 6

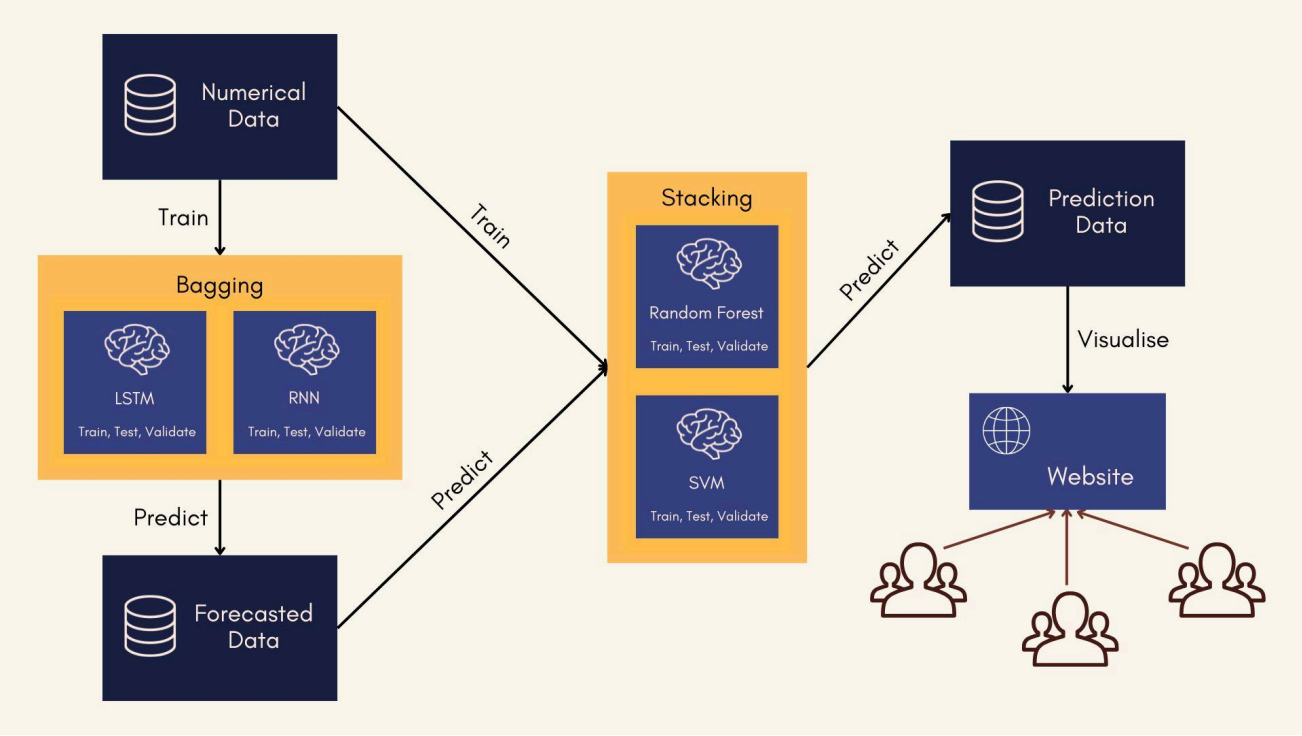
PROPOSED METHODOLOGY

6.1 Architecture

The system is majorly split into 3 sections, each flowing into the other -

1. Dataset Acquisition, Creation & Cleaning
2. Scale & Map Extraction
3. Scale Value Interpolation & Mapping
4. PFZ Prediction
5. PFZ Visualisation





6.2 Dataset Acquisition, Creation & Cleaning

The data has to be loaded into storage using the Copernicus Marine API. The required parameters are specified along with the target geographical region's coordinates, and the dataset is downloaded in the form of .nc files. A script converts the .nc files into .png image files, which is the required format for the project.

The images have to then be cleaned and enhanced to ensure uniformity across the dataset, and prepare it for further processing to extract required numerical information.

The output of this step is a clean, uniform dataset of satellite images of the required geographic region, ready for the next step.

6.3 Scale & Map Extraction

The raw satellite images of the previous step are now fed into the image processing module. The image processing module uses Contour Detection as well as a HSV colour filter.

The raw images are first split into 2 parts, one containing the map and the other containing the colour scale. Then, filters are applied on each part separately to identify the map region and the colour scale region accurately.

The output of this step is a dataset containing the exact map and scale regions of each satellite image, ready for interpolation in the next step.

6.4 Scale Value Interpolation & Mapping

The extracted map and scale regions of the satellite images are run through a FAISS K-Means model separately. Using the acquired cluster centroid values, each pixel on the map is accurately mapped to their respective colours on the colour scale.

The output of this step is a dataset containing numerical values of each pixel, for every image, corresponding to their respective parameter. This dataset will then be used to train the ML Model to predict the PFZs.

6.5 PFZ Prediction

The numerical dataset created is used to train 2 ensemble models to predict the final PFZ -

1. Bagging Ensemble Model
2. Stacking Ensemble Model

6.5.1 Bagging Ensemble Model

The bagging model consists of two DL models that are responsible for forecasting parameter values into the future. The models used are – LSTM and RNN.

- Load all the data derived from the image processing, split it into training and validation sets, and normalise the values using MinMaxScaler based on the training data.
- Convert the normalised data into 30-day sequences, with each sequence used to predict the next day's value, creating input and output pairs for both training and validation sets.
- Construct an LSTM model and RNN model, separately, with 50 units, ReLU activation, and a Dropout layer for regularisation, followed by a Dense output layer. Compile it using the Adam optimizer and Mean Squared Error loss function, and train them on the training sequences over 200 epochs.
- Use the trained models to individually predict values on the validation set, inverse transform the predictions to the original scale, and evaluate the model's performance using the Mean Squared Error metric. It finally forecasts all the parameter values into the future.

The forecasted parameters of both models are combined using a weighted average, which depends on the contribution of both models. This averaged forecast is then passed to the next ensemble model.

6.5.2 Stacking Ensemble Model

This ML model consists of an ensemble of two base learner models and one meta learner model which finally identify potential fishing zones based on the forecast results of the previous ensemble. The base learner models used are -

1. SVM

    - Load the data, select all the parameters as input, and label-encode the target variable into numerical format using LabelEncoder.
    - Split the dataset into training and testing subsets, ensuring stratified sampling to maintain class balance. Apply StandardScaler for normalising feature values in both training and testing data.

- Use SMOTE to oversample the minority class (Potential) to match its count in the training data. Apply RandomUnderSampler to reduce the majority class (Not potential) to a maximum of 10,000 samples, aiming for a balanced class distribution.
- Train a machine learning pipeline on the balanced training set. Evaluate the model's performance on the test set using a classification report for detailed metrics and visualise the results with a Confusion Matrix plot for accuracy assessment.
- Use the forecasted values from the previous ensemble as a new input set and predict the classes for all its entries.

2. Random Forest

- Load the data, select all the parameters as input, and label-encode the target variable into numerical format using LabelEncoder.
- Split the dataset into training and testing subsets, ensuring reproducibility with a fixed random state.
- Initialise a Random Forest Classifier with 100 estimators and fit it on the training data. Output model details like the number of estimators, max depth, minimum samples for split and leaf, and feature importances, providing insights into how each feature contributes to the classification.
- Visualise the importance of each feature and evaluate the model's performance on the test set by calculating accuracy and generating a classification report, which includes precision, recall, and F1-score for each class.
- Use the forecasted values from the previous ensemble as a new input set and predict the classes for all its entries.

The predicted classes from both the models are now passed as input features into the meta learner which is a trained Logistic Regression model. This model learns how to combine the results of the base learner appropriately so as to make a final prediction for each latitude and longitude, for each date.

6.6 PFZ Visualisation

The output labels of the prediction step will then be used to visualise the regions on a map, to make the results more user-friendly and readable. These visualisations will be hosted on a website for consumers to use for their benefit.

CHAPTER 7

IMPLEMENTATION AND PSEUDOCODE

7.1 Scale & Map Extraction

7.1.1 Algorithm

The algorithm is as follows:

1. Read raw map image
2. Split into left and right sections roughly
    a. Left section will contain the map
    b. Right section will contain the colour scale
3. Process each section separately using contour detection & HSV colour filtering

7.1.2 Pseudocode

The pseudocode is as follows:

1. Function `splitImage` : splits an image roughly into two parts - left (map) and right (scale).
    a. Read the image.
    b. Convert it to grayscale and detect edges.
    c. Find the largest contour (assumed to be the map).
    d. Calculate bounding coordinates for the map.
    e. Split the image at the map's right edge plus a small margin.
    f. Save the left and right cropped images as map and scale images.

2. Function `getScale` : extract the colour scale from the scale image.
    a. Load the scale image created by `splitImage`.
    b. Convert to HSV colour space.

c. Mask out non-white pixels to isolate the colour scale.
d. Find the largest contour (assumed to be the colour scale).
e. Add margins around the bounding box and crop.
f. Save the cropped scale image and delete the intermediate cropped image.

3. Function `getMap` : extract the map region from the map image.
    a. Load the map image created by `splitImage`.
    b. Convert to HSV colour space.
    c. Mask out non-white pixels to isolate the map.
    d. Find the largest contour (assumed to be the map).
    e. Create a mask from this contour and apply it to the image.
    f. Crop to the bounding rectangle of the map.
    g. Remove remaining white borders, if any.
    h. Save the cropped map image and delete the intermediate cropped image.

4. Function `getMapScale` : process an image and extract map and scale sections.
    a. Call `splitImage`, `getMap`, and `getScale` sequentially on the image.

5. Main Execution:
    a. Call `getMapScale` on all images in the directory.

7.2 Scale Value Interpolation & Mapping

7.2.1 Algorithm

The K-Means algorithm is as follows:

1. Randomly select K initial cluster centroids from the dataset
2. For each data point, calculate the distance to each centroid
3. Assign the data point to the nearest cluster
4. The centroids of each cluster are updated by calculating the mean of all the points assigned to that cluster

5. This is repeated until the centroids no longer change significantly or the maximum number of iterations is reached

The above K-Means algorithm is used with FAISS optimization as follows -

1. RGB values of the image data are reshaped into a suitable format for clustering
2. A FAISS index is created to facilitate efficient nearest neighbour searches
3. K-Means algorithm is configured with the desired parameters
4. The algorithm is trained and results in the identification of cluster centroids

Both of the above are combined to give -

1. Extract Scale Values Using OCR
    a. Read cropped colour scale image & convert to grayscale
    b. Use Tesseract OCR to parse & extract any numerical values found in the image
    c. Sort the extracted scale values in descending order to ensure they match the order in the physical layout of the scale

2. Detect Scale Label Positions and Colours
    a. Isolate the rightmost section of the scale_image where the scale labels are likely located
    b. To make the labels more distinct, invert pixel values so that text stands out against the background
    c. Convert pixel values to grayscale by averaging across colour channels, and then invert pixel intensity.
    d. Sum the pixel intensities along the vertical axis. Peaks in this summed intensity array indicate the positions of labels.
    e. For each detected label position, extract the RGB colour from pixels located slightly to the left of each label. Average these pixels if needed to capture a representative colour for each label.

3. Cluster Image Colours Using K-Means with FAISS optimization

4. Retrieve centroids from the trained model

5. Map labels to closes centroid colours
    a. Calculate the Euclidean distance to each colour centroid from KMeans
    b. Identify the closest colour centroid to each label colour based on minimum Euclidean distance & store the scale value along with the associated centroid colour

6. Generate interpolated values for centroids
    a. Find the two nearest scale values based on colour similarity for each centroid
    b. Calculate a weighted average of the two nearest scale values based on their distances to the centroid colour
    c. Store each centroid colour along with its interpolated scale value

7. Match map image colours to scale centroids
    a. Repeat the clustering process on the map image to obtain centroids
    b. For each map colour centroid, calculate the Euclidean distance to all scale centroids
    c. For each map centroid, find the nearest scale centroid based on colour similarity
    d. Use the interpolated scale value to assign a scale value to each map colour centroid

8. Store the mapping of map centroids to scale values

7.2.2 Pseudocode

The pseudocode is as follows:

1. Crop and Preprocess Scale Image
    a. Load image
    b. Crop image to specified coordinates (scale region)
    c. Convert cropped scale image to grayscale for OCR

2. Extract Scale Values Using OCR
    a. Run OCR on grayscale image
    b. Extract and parse numerical values

c. Sort scale values in descending order

3. Detect Scale Label Positions and Colours
    a. Convert cropped image to NumPy array
    b. Extract rightmost region (label area) and invert colours
    c. Sum pixel intensities along vertical axis
    d. Find peaks in summed values to locate label positions
    e. For each detected position: Extract RGB colour to the left of the label

4. Cluster Image Colours Using KMeans
    a. Flatten RGB data from cropped image
    b. Configure KMeans with k clusters and initialise FAISS index
    c. Train KMeans on RGB data to obtain centroids
    d. Retrieve cluster centroids

5. Map Labels to Closest Centroid Colours
    a. For each scale label colour:
        i. Find the closest centroid (Euclidean distance)
        ii. Map scale value to closest centroid RGB

6. Generate Interpolated Values for Centroids
    a. For each centroid:
        i. Find two nearest scale values
        ii. Interpolate between the two nearest scale values based on colour distance
        iii. Store interpolated scale value for each centroid

7. Match Map Image Colours to Scale Centroids
    a. Cluster RGB data from map image to obtain map centroids
    b. For each map centroid:
        i. Find nearest scale centroid by Euclidean distance
        ii. Map nearest scale centroid RGB and corresponding scale value

8. Output Results
    a. For each map centroid:
        i. Store centroid RGB, nearest scale centroid RGB, and interpolated scale value

7.3 PFZ Prediction

7.3.1 Pseudocode

The pseudocode of our Bagging Model is as follows:

1. Function `trainModels`
    a. Load the training dataset and preprocess the data.
    b. Define the LSTM Model architecture, compile the model and train it on the data.
    c. Define the RNN Model architecture, compile the model and train it on the data.
    d. Save the models.

2. Function `makePredictions`
    a. Load the saved models from function `trainModels`
    b. Load the input dataset for forecasting and preprocess it
    c. Use the LSTM and RNN to predict the future values individually.
    d. Return these predictions.

3. Function `baggingEnsemble`
    a. Element wise average of LSTM and RNN predictions is computed.
    b. Return the average forecasted data.

4. Function `trainModels`
    a. Load the training dataset and preprocess the data.
    b. Define the LSTM Model architecture, compile the model and train it on the data.
    c. Define the RNN Model architecture, compile the model and train it on the data.
    d. Save the models.

5. Function `makePredictions`

a. Load the saved models from function `trainModels`
b. Load the input dataset for forecasting and preprocess it

The pseudocode of our Stacking Model is as follows:

1. Function `trainBaseModels`
    a. Load the training dataset and preprocess the data.
    b. Define the Random Forest with specified parameters (number of estimators, max depth), fit it on the training data
    c. Define the SVM model with appropriate kernel and hyperparameters, fit it on the training data.
    d. Save the models.

2. Function `generateBasePredictions`
    a. Load the saved models.
    b. Use the SVM and Random Forest model to predict class probabilities or labels.
    c. Concatenate the predictions as new features.
    d. Return the new feature set and original labels

3. Function `trainMetaLearner`
    a. Accept the new feature set and labels from `generateBasePredictions`
    b. Define the Logistic Regression with specified hyperparameters, fit it on the stacked data.
    c. Save the meta-learner model

4. Function `stackingModelPredict`
    a. Load the base models and the meta-learner models.
    b. Load the new dataset whose predictions are needed, i.e, the output of the previous bagging model.
    c. Generate the base model predictions.
    d. Concatenate these predictions as new features for the meta-learner.

e. Use the trained Logistic Regression model to make final predictions.

f. Save the predictions to a new CSV file.

CHAPTER 8

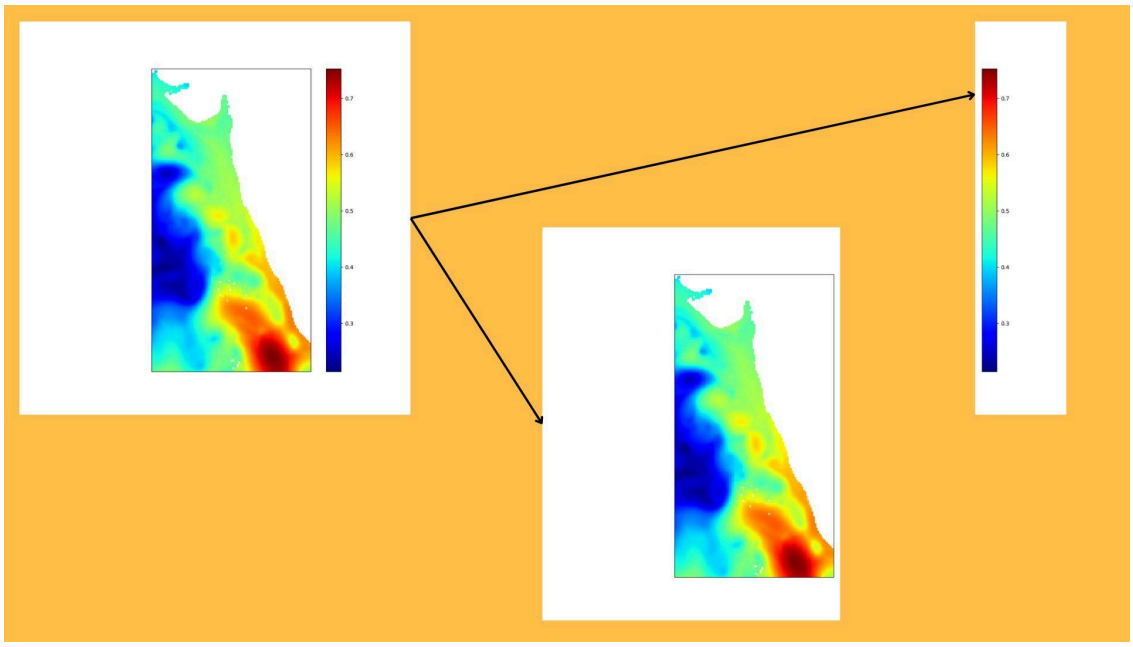
RESULTS AND DISCUSSION

8.1 Scale & Map Extraction

8.1.1 Separation of Map & Colour Scale Regions

The raw satellite image is first split into the regions containing only the map and the colour scale.

A simple contour detection method was used to detect the map's edge first, using which, the image was divided and stored separately as intermediate results.

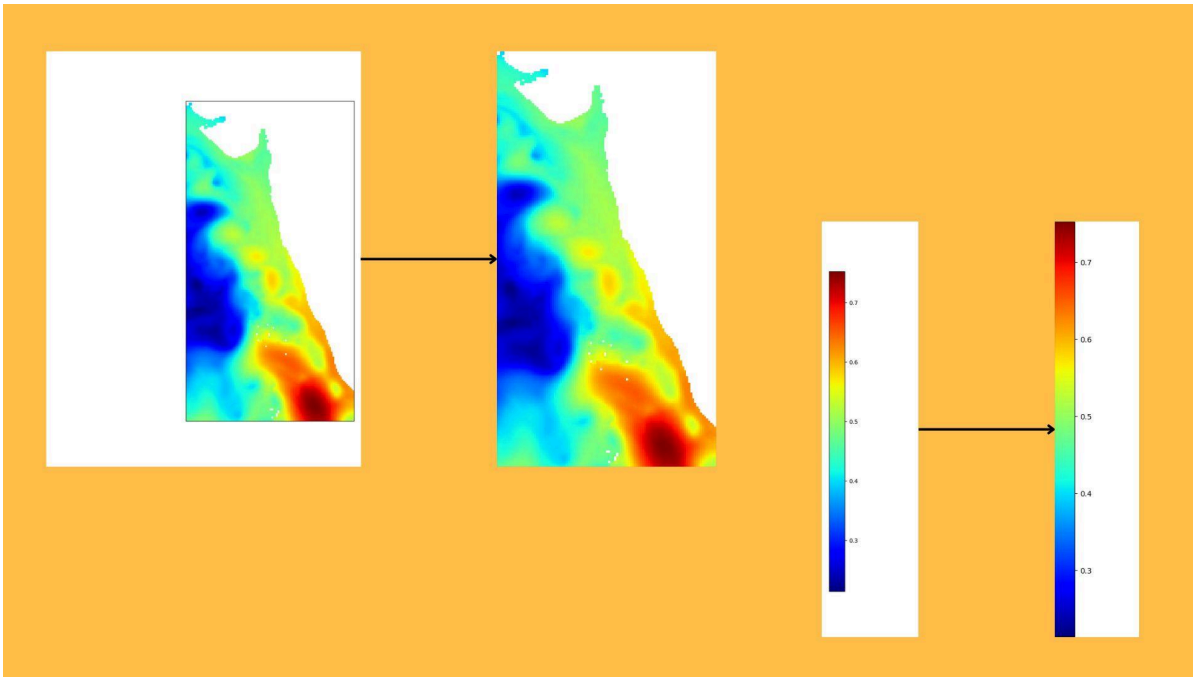


8.1.2 Exact Extraction of Map and Colour Scale

The roughly separated map and colour scale images are then run through -

1. Contour Detection
2. HSV Filter

The result of this step, is an exact separation of the map and the colour scale from their backgrounds, that can be used in the Interpolation ML Model to extract numerical data.



8.2 Scale Value Interpolation ML Model

8.2.1 Introduction

The next step after separation of the map and the colour scale, is to map each pixel of the map image, to its respective numerical value on the colour scale. While there are manual ways to do this, a machine learning approach was deemed most suitable not only for novelty, but also for accuracy and

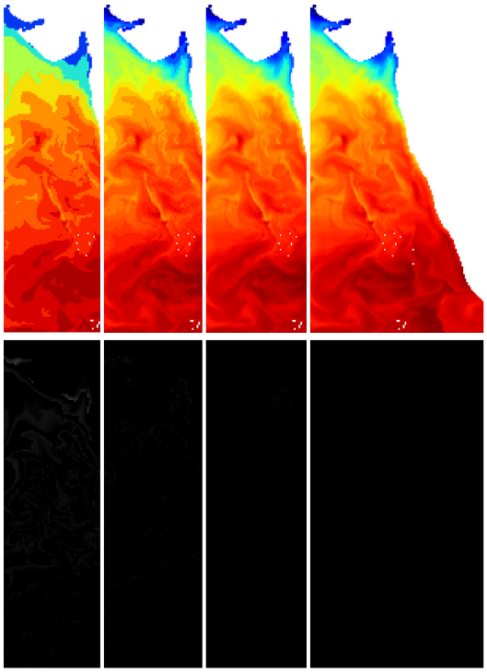
efficiency, since the prediction model would need a robust dataset for training to achieve optimal results.

8.2.2 Exploration

Multiple ML Models were explored to select the most optimal one for the use case of this project.

8.2.2.1 Simple K-Means Clustering

The image is first converted into an array of values representing the pixels of the image. SciKitLearn's KMeans model is used to cluster the map and the scale.



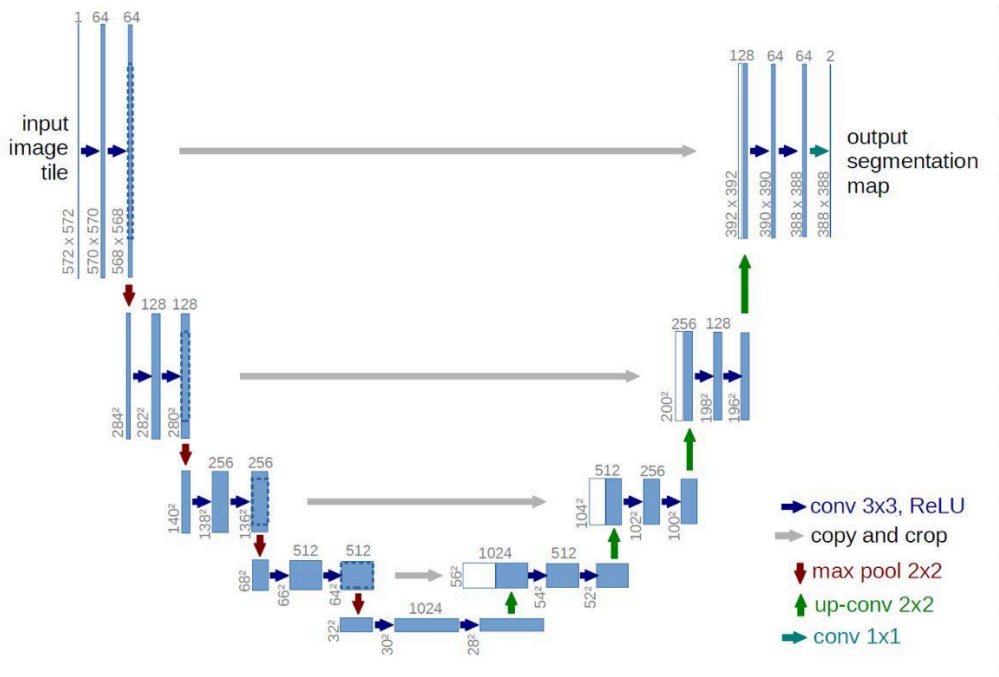
```
Image matching Error between the two images: 0.08317148773617668
```

In conclusion, while the simple K-Means model works very efficiently with an error of only 0.08, it takes longer to train compared to K-Means with FAISS optimization.

8.2.2.3 U-Net

Requirement: Pixel-wise analysis on both the localization accuracy and the abstraction ability

Solution: Pixel-wise regression using U-Net



The model was trained with the following parameters -

1. Optimizer: ADAM
2. Learning Rate: 0.0001
3. Loss Metric: MSE

Layerwise model details -

1. Layer: Input
    a. Shape: (256, 256, 3)
    b. Description: Accepts input images of size 256x256 pixels with 3 colour channels (RGB).

Layer (type)	Output Shape	Param #	Connected to
input_layer_6 (InputLayer)	(None, 256, 256, 3)	0	-

2. Layer: Encoders
    a. 8 convolutional layers (2D convolutions)
    b. 4 max-pooling layers
    c. Block 1 : 64 filters
    d. Block 2: 128 filters
    e. Block 3: 256 filters
    f. Block 4: 512 filters
    g. Bottleneck: 1024 filter

conv2d_139 (Conv2D)	(None, 256, 256, 64)	1,792	input_layer_6[0]…
conv2d_140 (Conv2D)	(None, 256, 256, 64)	36,928	conv2d_139[0][0]
max_pooling2d_24 (MaxPooling2D)	(None, 128, 128, 64)	0	conv2d_140[0][0]
conv2d_141 (Conv2D)	(None, 128, 128, 128)	73,856	max_pooling2d_24…
conv2d_142 (Conv2D)	(None, 128, 128, 128)	147,584	conv2d_141[0][0]
max_pooling2d_25 (MaxPooling2D)	(None, 64, 64, 128)	0	conv2d_142[0][0]
conv2d_143 (Conv2D)	(None, 64, 64, 256)	295,168	max_pooling2d_25…
conv2d_144 (Conv2D)	(None, 64, 64, 256)	590,080	conv2d_143[0][0]
max_pooling2d_26 (MaxPooling2D)	(None, 32, 32, 256)	0	conv2d_144[0][0]
conv2d_145 (Conv2D)	(None, 32, 32, 512)	1,180,160	max_pooling2d_26…
conv2d_146 (Conv2D)	(None, 32, 32, 512)	2,359,808	conv2d_145[0][0]
max_pooling2d_27 (MaxPooling2D)	(None, 16, 16, 512)	0	conv2d_146[0][0]
conv2d_147 (Conv2D)	(None, 16, 16, 1024)	4,719,616	max_pooling2d_27…
conv2d_148 (Conv2D)	(None, 16, 16, 1024)	9,438,208	conv2d_147[0][0]

3. Layer: Decoders
    a. 8 convolutional layers (2D convolutions)
    b. 4 upsampling layers (2D upsampling)
    c. 4 concatenations
    d. Block 1 : 512 filters
    e. Block 2: 256 filters
    f. Block 3: 128 filters
    g. Block 4: 64 filters

up_sampling2d_26 (UpSampling2D)	(None, 128, 128, 256)	0	conv2d_154[0][0]
conv2d_155 (Conv2D)	(None, 128, 128, 128)	131,200	up_sampling2d_26…
concatenate_26 (Concatenate)	(None, 128, 128, 256)	0	conv2d_142[0][0], conv2d_155[0][0]
conv2d_156 (Conv2D)	(None, 128, 128, 128)	295,040	concatenate_26[0…
conv2d_157 (Conv2D)	(None, 128, 128, 128)	147,584	conv2d_156[0][0]
up_sampling2d_27 (UpSampling2D)	(None, 256, 256, 128)	0	conv2d_157[0][0]
conv2d_158 (Conv2D)	(None, 256, 256, 64)	32,832	up_sampling2d_27…
concatenate_27 (Concatenate)	(None, 256, 256, 128)	0	conv2d_140[0][0], conv2d_158[0][0]
conv2d_159 (Conv2D)	(None, 256, 256, 64)	73,792	concatenate_27[0…
conv2d_160 (Conv2D)	(None, 256, 256, 64)	36,928	conv2d_159[0][0]
conv2d_161 (Conv2D)	(None, 256, 256, 772)	50,180	conv2d_160[0][0]

4. Layer: Output - 2D Convolution
    a. Filters: 1
    b. Kernel Size: (1, 1)
    c. Activation: Linear
    d. Produces a single-channel output (for regression) with a linear activation function.

```
Total params: 31,081,860 (118.57 MB)

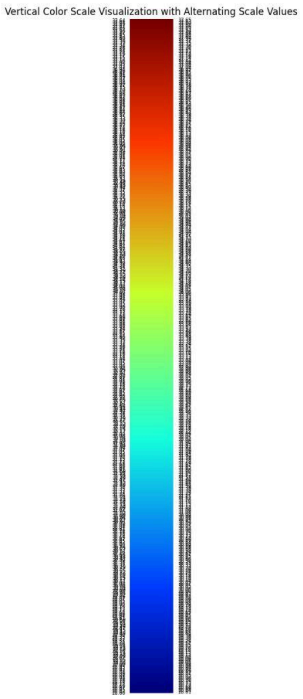
Trainable params: 31,081,860 (118.57 MB)
```

The scale image is analysed at specific detected positions to extract RGB values. These positions are along the vertical centre of the scale image. A lower bound and upper bound value is calculated to extend the scale beyond the detected positions using linear extrapolation. These bounds are specified with the RGB values for either side of the scale.

```
28.6: [  0   0 120]
30: [  0  59 255]
32: [ 22 255 225]
34: [202 255  44]
36: [255  57   0]
37.65: [120   0   0]
```

For each segment between detected positions, the function calculates the number of pixels and interpolates both the scale values and RGB colours. The interpolation is linear, where for each pixel between two positions, scale values and RGB colours are computed based on their respective ratios.

```
Pixel: 102, Scale Value: 29.80, RGB: (0, 50, 235)
Pixel: 103, Scale Value: 29.81, RGB: (0, 51, 236)
Pixel: 104, Scale Value: 29.82, RGB: (0, 51, 237)
Pixel: 105, Scale Value: 29.84, RGB: (0, 52, 239)
Pixel: 106, Scale Value: 29.85, RGB: (0, 52, 240)
Pixel: 107, Scale Value: 29.86, RGB: (0, 53, 241)
Pixel: 108, Scale Value: 29.87, RGB: (0, 53, 242)
Pixel: 109, Scale Value: 29.88, RGB: (0, 54, 243)
Pixel: 110, Scale Value: 29.89, RGB: (0, 54, 244)
Pixel: 111, Scale Value: 29.91, RGB: (0, 55, 245)
Pixel: 112, Scale Value: 29.92, RGB: (0, 55, 247)
Pixel: 113, Scale Value: 29.93, RGB: (0, 56, 248)
Pixel: 114, Scale Value: 29.94, RGB: (0, 56, 249)
Pixel: 115, Scale Value: 29.95, RGB: (0, 57, 250)
Pixel: 116, Scale Value: 29.96, RGB: (0, 57, 251)
Pixel: 117, Scale Value: 29.98, RGB: (0, 58, 252)
Pixel: 118, Scale Value: 29.99, RGB: (0, 58, 253)
Pixel: 119, Scale Value: 30.00, RGB: (0, 59, 255)
Pixel: 120, Scale Value: 30.01, RGB: (0, 60, 254)
Pixel: 121, Scale Value: 30.02, RGB: (0, 61, 254)
Pixel: 122, Scale Value: 30.04, RGB: (0, 62, 254)
Pixel: 123, Scale Value: 30.05, RGB: (0, 63, 254)
Pixel: 124, Scale Value: 30.06, RGB: (0, 64, 254)
Pixel: 125, Scale Value: 30.07, RGB: (0, 65, 253)
Pixel: 126, Scale Value: 30.08, RGB: (0, 67, 253)
Pixel: 127, Scale Value: 30.09, RGB: (1, 68, 253)
Pixel: 128, Scale Value: 30.11, RGB: (1, 69, 253)
Pixel: 129, Scale Value: 30.12, RGB: (1, 70, 253)
```



The result was an accuracy of 96.87% over 10 epochs, taking 32.7 seconds to train.

```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 10s 10s/step - accuracy: 0.000
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.0000e
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.0010
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.0298
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.9060
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.9572
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.9642
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.9648
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.9679
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.9687
```

```
[33.08 33.08 33.08 ... 33.08 33.08 33.08]
[33.08 33.08 33.08 ... 33.08 33.08 33.08]
...
[34.23 34.12 34.09 ... 34.2  34.2  34.28]
[34.23 34.12 34.09 ... 34.2  34.2  34.28]
[34.33 34.23 34.19 ... 34.28 34.28 34.41]]
```

8.2.2.6 Other Models

Some other models that were explored, but not used in the project due to inefficiency, cost of training or other such reasons are -

1. DBSCAN Clustering
2. Spectral Clustering
3. Contour Detection
4. ResNet

8.2.3 FAISS K-Means Clustering

Finally, K-Means clustering with FAISS optimization was decided to be used due to its increased efficiency without sacrificing on accuracy.

Hyper-parameters used -

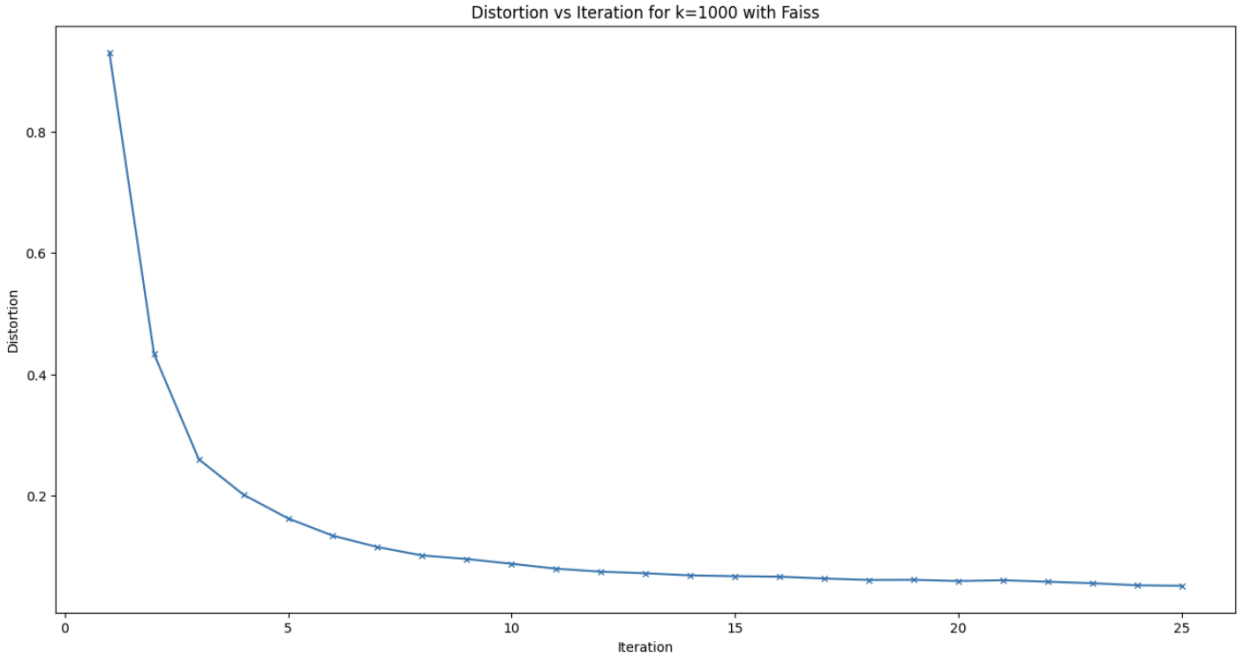
1. Number of clusters: 1000
2. L2 FAISS index used
3. kmeans.niter = 20

Means of validating goodness of clustering via plotting the Distortion curve for K-Means clustering with FAISS

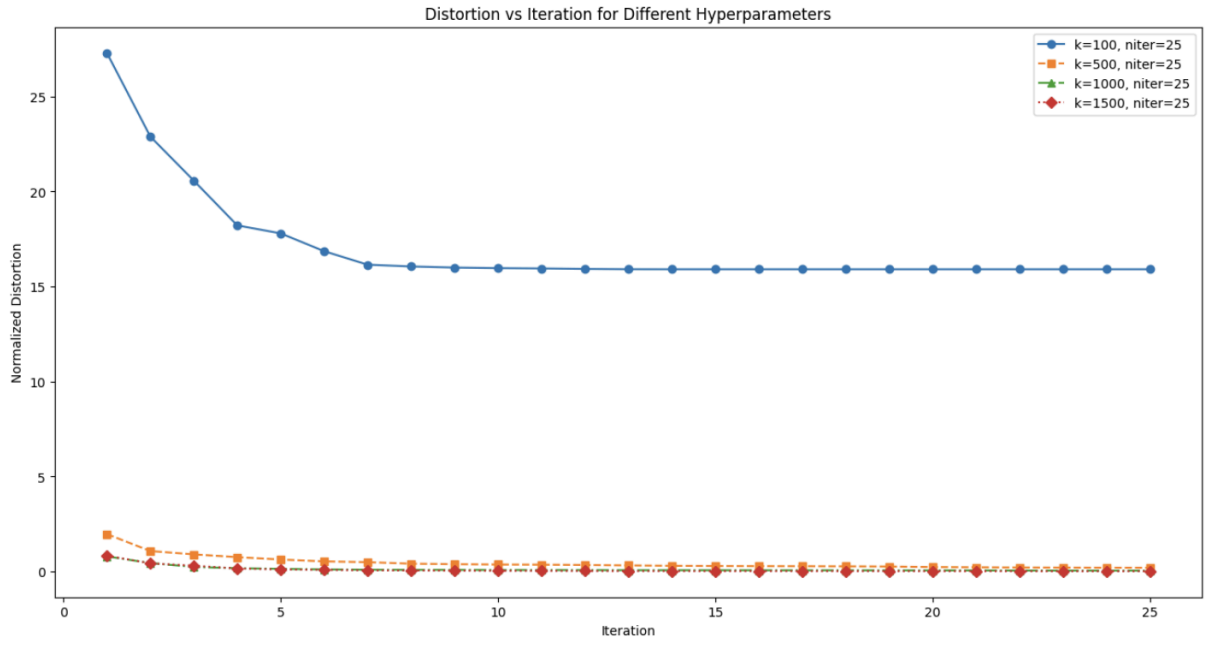
Plotting normalised distortion to measure clustering goodness on the right scale.

```
normalized_distortion = total_distortion / (filtered_data.shape[0] * d)
```

```
data shape: 266086
Total distortion: 5182873.0
```



Hyperparameter tuning:



8.3 Prediction ML Model

8.3.1 Introduction

The final CSV produced in the previous step consisting of all the parameter values is given to two ensemble models – a forecasting bagging model and a classifying stacking model.

8.3.1 Bagging model

The bagging model consists of an LSTM and RNN, that study the underlying patterns of the oceanic data and forecast the parameters into the future.

8.3.1.1 LSTM

LSTM Layer:

- Output Shape: (None, 50) - 50 units in the LSTM layer.
- Parameters: 10,400 - These are the parameters for the LSTM layer, which include weights for the input, recurrent, and bias terms.
    - Weights for input to LSTM: (input_shape[1] * units) = (X_train.shape[1] * 50)
    - Weights for recurrent connections: (units * units) = (50 * 50)
    - Bias terms: 50

Dropout Layer:

- Output Shape: (None, 50) - The same output shape as the LSTM layer.
- Parameters: 0 - Dropout does not add any learnable parameters. It just randomly sets a fraction of inputs to 0 during training.

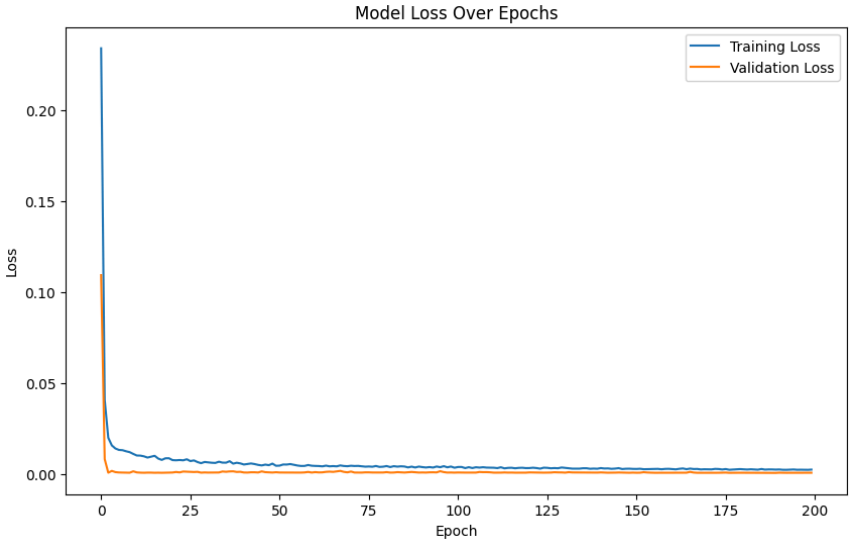
Dense Layer:

- Output Shape: (None, 1) - The final output shape is a single value, which is a regression output.
- Parameters: 51 - These are the parameters for the Dense layer, which consist of:

- Weights: 50 * 1 = 50 (Each input from the LSTM layer is connected to one output node)
- Bias term: 1 (One bias term for the output node)

```
Epoch 1/200
19/19 [==============================] - 1s 10ms/step - loss: 0.2065 - val_loss: 0.0955
Epoch 2/200
19/19 [==============================] - 0s 5ms/step - loss: 0.1735 - val_loss: 0.0705
Epoch 3/200
19/19 [==============================] - 0s 5ms/step - loss: 0.1380 - val_loss: 0.0474
Epoch 4/200
19/19 [==============================] - 0s 5ms/step - loss: 0.1036 - val_loss: 0.0253
Epoch 5/200
19/19 [==============================] - 0s 8ms/step - loss: 0.0662 - val_loss: 0.0077
Epoch 6/200
19/19 [==============================] - 0s 6ms/step - loss: 0.0371 - val_loss: 2.7663e-04
Epoch 7/200
19/19 [==============================] - 0s 6ms/step - loss: 0.0178 - val_loss: 0.0082
Epoch 8/200
19/19 [==============================] - 0s 6ms/step - loss: 0.0174 - val_loss: 0.0066
Epoch 9/200
19/19 [==============================] - 0s 6ms/step - loss: 0.0142 - val_loss: 0.0066
Epoch 10/200
19/19 [==============================] - 0s 6ms/step - loss: 0.0157 - val_loss: 0.0073
Epoch 11/200
19/19 [==============================] - 0s 5ms/step - loss: 0.0149 - val_loss: 0.0073
Epoch 12/200
19/19 [==============================] - 0s 5ms/step - loss: 0.0142 - val_loss: 0.0080
Epoch 13/200
...
Epoch 199/200
19/19 [==============================] - 0s 5ms/step - loss: 0.0054 - val_loss: 0.0147
Epoch 200/200
19/19 [==============================] - 0s 5ms/step - loss: 0.0049 - val_loss: 0.0128
```

```
2/2 [==============================] - 0s 3ms/step
Validation Mean Squared Error: 8.921964484370445e-05
```



The LSTM finally makes forecasts for each of the parameters into the future.

```
1/1 [==============================] - 0s 11ms/step
1/1 [==============================] - 0s 10ms/step
1/1 [==============================] - 0s 11ms/step
1/1 [==============================] - 0s 11ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 10ms/step
1/1 [==============================] - 0s 12ms/step
1/1 [==============================] - 0s 10ms/step
1/1 [==============================] - 0s 10ms/step
1/1 [==============================] - 0s 12ms/step
...
 [0.38130635]
 [0.38114226]
 [0.38097692]
 [0.3808122 ]]
```

8.3.1.2 RNN

SimpleRNN Layer:

- Output Shape: (None, 50)
- Parameters:
    - Input weights: input_dim * units = X_train.shape[1] * 50
    - Recurrent weights: units * units = 50 * 50

- Bias: 50

- Total for SimpleRNN: Total = (Xtrain.shape[1] ∗ 50) + (50 ∗ 50) + 50

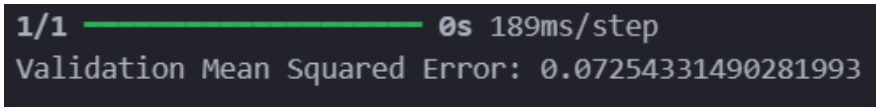
Dropout Layer:

- Parameters: 0 (no trainable parameters)

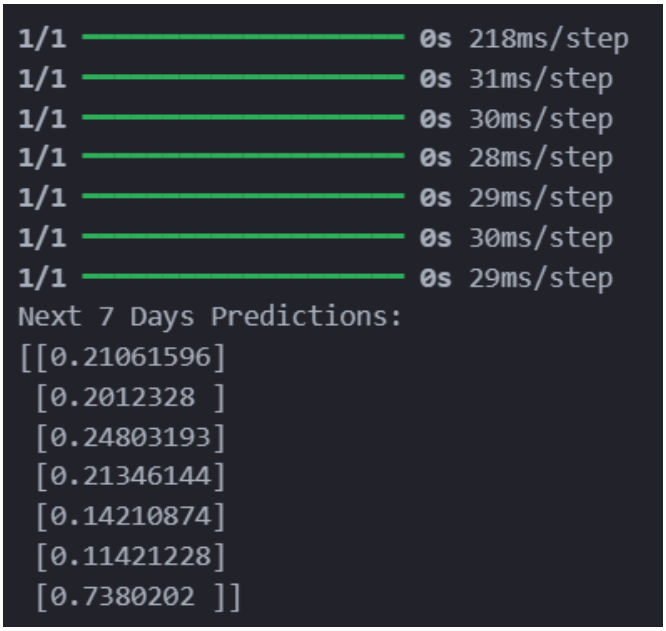
Dense Layer:

- Output Shape: (None, 1)
- Parameters: 50 * 1 + 1 = 51

```
Epoch 1/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 7s 45ms/step - loss: 0.0397 - val_loss: 0.0455
Epoch 2/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0419 - val_loss: 0.0424
Epoch 3/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0267 - val_loss: 0.0410
Epoch 4/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0267 - val_loss: 0.0391
Epoch 5/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0339 - val_loss: 0.0382
Epoch 6/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0294 - val_loss: 0.0381
Epoch 7/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0278 - val_loss: 0.0391
Epoch 8/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0240 - val_loss: 0.0408
Epoch 9/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0226 - val_loss: 0.0445
Epoch 10/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0214 - val_loss: 0.0458
Epoch 11/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0185 - val_loss: 0.0540
Epoch 12/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0207 - val_loss: 0.0500
Epoch 13/200
...
Epoch 199/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0019 - val_loss: 0.0599
Epoch 200/200
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0016 - val_loss: 0.0571
```



The RNN finally makes forecasts for each of the parameters into the future.



8.3.1.3 Final result

The forecasted outputs of the LSTM and RNN are combined using a weighted average. This is done for all parameters over the next 7 days, for every latitude and longitude. The outputs are stored in a final CSV, which is used to train the next ensemble model.

latitude	longitude	time	chl	o2	phyc	so	thetao	zos
6.5	68.25	31-10-2024	0.188342	198.914	1.483613	36.36431	29.13567	0.300458
6.5	68.25	01-11-2024	0.204747	199.7261	1.510267	36.23544	29.48497	0.306358
6.5	68.25	02-11-2024	0.19337	199.8932	1.514731	36.18546	29.56832	0.317945
6.5	68.25	03-11-2024	0.17442	199.7103	1.504404	36.14688	29.52712	0.316382
6.5	68.25	04-11-2024	0.168219	199.7375	1.462386	36.13311	29.26592	0.313587
6.5	68.25	05-11-2024	0.162896	199.4859	1.44127	36.13853	29.28252	0.321955
6.5	68.25	06-11-2024	0.157161	199.324	1.422941	36.21814	29.59569	0.324245
6.5	68.25	07-11-2024	0.150067	199.2483	1.432599	36.41974	29.99771	0.336142
6.5	68.25	08-11-2024	0.142749	199.2529	1.464987	36.39401	29.96711	0.33666
6.5	68.25	09-11-2024	0.135324	199.375	1.479703	36.29061	29.97479	0.337267
6.5	68.25	10-11-2024	0.133979	199.3977	1.514738	36.18423	30.06043	0.33945
6.5	68.25	11-11-2024	0.13106	199.1669	1.526095	36.16505	29.94552	0.352593
6.5	68.25	12-11-2024	0.127448	199.0271	1.508238	36.17024	29.85631	0.361394
6.5	68.25	13-11-2024	0.127481	198.9624	1.527836	36.18318	30.03077	0.342136
6.5	68.5	31-10-2024	0.190686	198.8432	1.49655	36.45586	29.18086	0.306902
6.5	68.5	01-11-2024	0.198118	199.4754	1.498958	36.29214	29.31497	0.312107
6.5	68.5	02-11-2024	0.194299	199.7965	1.509133	36.28217	29.52581	0.32624
6.5	68.5	03-11-2024	0.184585	199.7332	1.507356	36.23179	29.50856	0.322746
6.5	68.5	04-11-2024	0.181256	199.8413	1.47944	36.24224	29.28221	0.318073
6.5	68.5	05-11-2024	0.173745	199.5829	1.469626	36.20548	29.25302	0.326727
6.5	68.5	06-11-2024	0.163495	199.392	1.474611	36.19327	29.48952	0.329485
6.5	68.5	07-11-2024	0.156268	199.3693	1.487573	36.23877	29.72211	0.342909
6.5	68.5	08-11-2024	0.149716	199.428	1.490818	36.4371	30.03007	0.342231
6.5	68.5	09-11-2024	0.141157	199.5143	1.49272	36.38681	29.98706	0.341437

8.3.2 Stacking model

The stacking model consists of 2 base learner models - a SVM and a Random Forest Model, that train using the original image processing numerical data. After training, they make predictions on the output of the Bagging model. These predictions are the input features for a meta learner model, which combines them appropriately to give a final prediction.

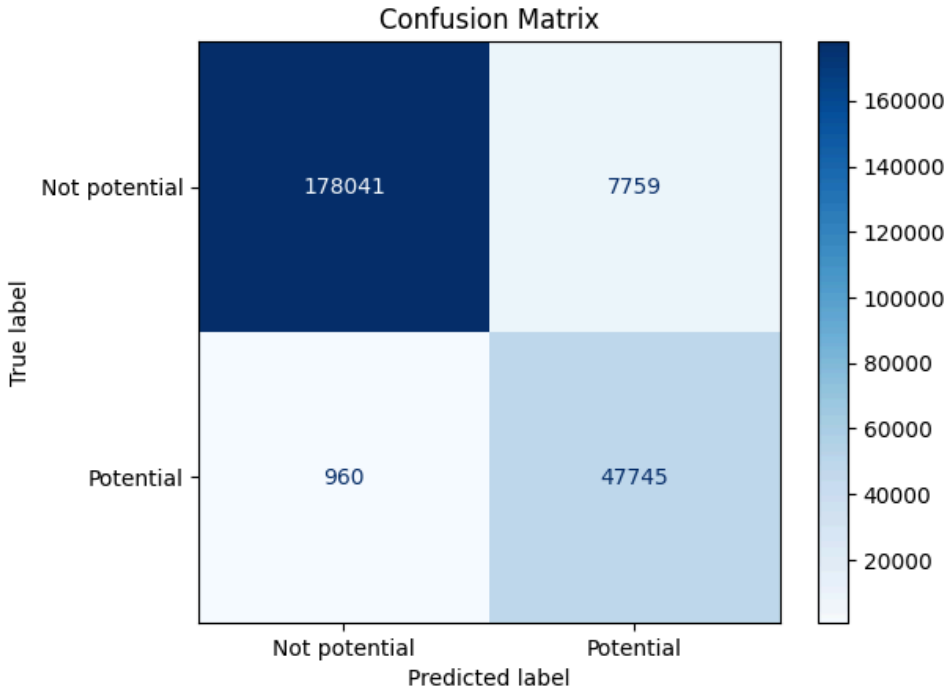
8.3.2.1 SVM

SVC Model:

- Kernel: linear
- C: 1
- Support Vectors: Model's complexity depends on the data and C.

Classification Report and Metrics:

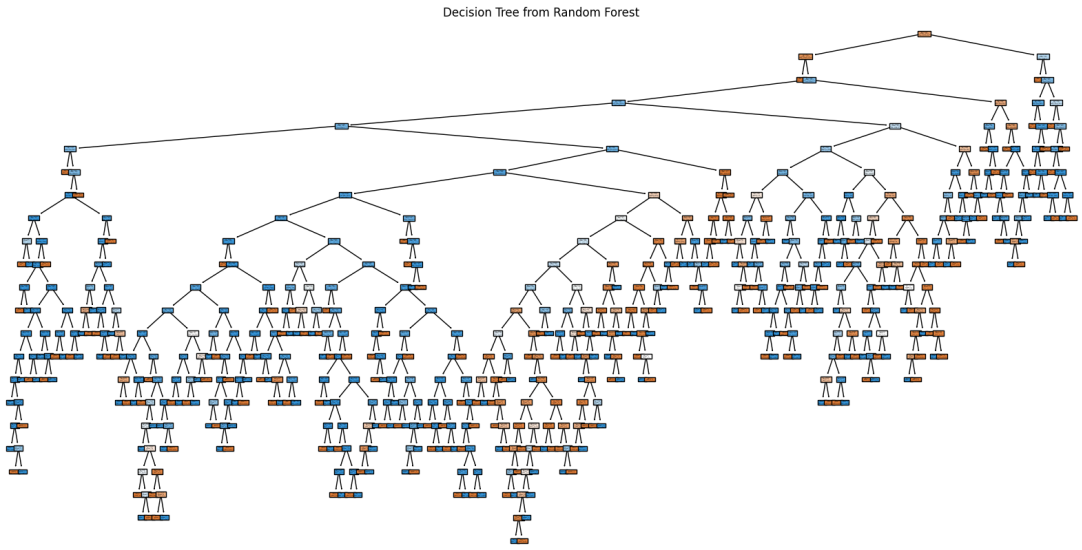
	precision	recall	f1-score	support
Not potential	0.99	0.96	0.98	185800
Potential	0.86	0.98	0.92	48705
accuracy			0.96	234505
macro avg	0.93	0.97	0.95	234505
weighted avg	0.97	0.96	0.96	234505



8.3.2.2 Random Forest Model

RandomForestClassifier:

- n_estimators: 100
- max_depth: None (unrestricted depth)
- min_samples_split: 2
- min_samples_leaf: 1
- Random State: 42



Accuracy: 100.00%

	precision	recall	f1-score	support
Not potential	1.00	1.00	1.00	185771
Potential	1.00	1.00	1.00	48734
accuracy			1.00	234505
macro avg	1.00	1.00	1.00	234505
weighted avg	1.00	1.00	1.00	234505

8.3.2.3 Logistic Regression Model

This is the meta-learner model which makes the final predictions in the form of a csv with a category label as the output.

latitude	longitude	time	chl	o2	phyc	so	thetao	zos	Category
9.5	73.25	01-11-2024	0.142954	199.7974	1.627376	35.68472	29.91204	0.355095	Not potential
9.5	73.5	01-11-2024	0.156694	200.2908	1.718696	35.61809	29.90584	0.358641	Not potential
9.5	73.75	01-11-2024	0.158498	200.6793	1.648286	35.06387	29.83211	0.366264	Not potential
9.5	74	01-11-2024	0.161221	200.8167	1.623639	35.04378	29.77104	0.369324	Not potential
9.5	74.25	01-11-2024	0.183103	201.2558	1.694523	35.11157	29.71396	0.373801	Not potential
9.5	74.5	01-11-2024	0.207535	201.9097	1.781495	35.24122	29.65866	0.385141	Potential
9.5	74.75	01-11-2024	0.216069	202.3066	1.862922	34.99992	29.67596	0.404395	Potential
9.5	75	01-11-2024	0.232376	203.065	2.07816	34.93651	29.77696	0.417111	Potential
9.5	75.25	01-11-2024	0.249363	203.6117	2.206761	34.8732	29.79654	0.423217	Potential
9.5	75.5	01-11-2024	0.28742	204.4535	2.350329	34.92254	29.77633	0.428157	Potential
9.5	75.75	01-11-2024	0.42529	207.0138	3.15329	34.98536	29.83381	0.433287	Potential
9.5	76	01-11-2024	0.577036	211.3484	3.847199	34.93451	29.80456	0.43262	Potential
9.5	76.25	01-11-2024	1.206521	213.7267	6.617706	34.77293	29.95621	0.433099	Potential
9.75	68.25	01-11-2024	0.143617	200.1319	1.681053	36.73132	30.30826	0.268295	Not potential
9.75	68.5	01-11-2024	0.141984	200.0645	1.687206	36.69828	30.27364	0.25726	Not potential
9.75	68.75	01-11-2024	0.14426	199.8814	1.737778	36.62363	30.31569	0.251295	Not potential
9.75	69	01-11-2024	0.149923	199.9205	1.813248	36.22244	30.26663	0.24561	Not potential
9.75	69.25	01-11-2024	0.15148	200.0176	1.843065	35.43479	30.20993	0.242144	Not potential
9.75	69.5	01-11-2024	0.15033	200.0616	1.846252	35.28413	30.28136	0.236082	Not potential
9.75	69.75	01-11-2024	0.148488	200.1005	1.823423	35.25943	30.33802	0.230631	Not potential
9.75	70	01-11-2024	0.144605	200.3714	1.757181	35.2846	30.39551	0.225682	Not potential
9.75	70.25	01-11-2024	0.146239	200.4166	1.76013	35.27616	30.39811	0.220249	Not potential
9.75	70.5	01-11-2024	0.147909	200.2186	1.785061	35.23815	30.3623	0.217279	Not potential

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

To conclude our work on the project, we have -

- Explored various methods to process satellite images
- Explored various ML models to extract pixel-wise values from the satellite images and to forecast and classify PFZs
- Built a ground-up system involving the best models and techniques and a frontend to visualise the results acquired

This project has potential to be used by various government agencies, private entities as well as individuals not only for commercial but also for recreational fishing activities. The project is novel in the way that the results are visualised in a very user-friendly manner where little to no prior knowledge is required to understand and use them. Additionally, future works relating to Potential Fishing Zone Prediction and Oceanic Satellite Image Processing can refer to the architecture built and research done in this project.

9.2 Future Work

The future work planned mainly revolves around -

1. Improving and refining our U-Net model for image processing to extract numerical data from satellite images
2. Increasing the area of coverage for PFZ prediction to cover the east coastline of India
3. Increasing the variety of commercial and recreational fish species for PFZ prediction